
DATA SCIENCE

Connaissance client en Assurance Auto

Florian Pincemin, François Beaune, Zié Coulibaly



Introduction

Contexte & Business Problem

Dans un marché de l'assurance en constante évolution, la diversification des produits et l'adaptation aux besoins des clients deviennent des enjeux stratégiques majeurs. Une compagnie d'assurances, désireuse d'étendre ses activités, a entrepris une initiative visant à proposer une assurance automobile à ses nouveaux assurés. Dans ce cadre, une campagne marketing a été menée auprès de plusieurs dizaines de milliers de clients, permettant de collecter des données précieuses sur leur intérêt pour ce produit.

Cette démarche repose sur l'hypothèse qu'un modèle de prédiction efficace pourrait maximiser l'impact de telles campagnes en identifiant les clients les plus susceptibles de souscrire à une assurance auto. En exploitant les données clients disponibles, l'objectif est de résoudre une double problématique : optimiser l'attribution des ressources commerciales tout en améliorant l'expérience client grâce à des propositions personnalisées.

Toutefois, la complexité de cette tâche réside dans la nature hétérogène des données collectées, qui incluent des informations personnelles, comportementales et historiques, ainsi que dans la nécessité d'atteindre un compromis entre précision et rappel dans les prédictions. Ces défis soulignent l'importance d'une approche rigoureuse et méthodique pour le développement du modèle prédictif.

Objectifs

L'objectif principal de ce projet est de concevoir un modèle de machine learning capable de prédire, avec un haut degré de fiabilité, l'intérêt d'un assuré pour souscrire une assurance automobile. Ce modèle sera utilisé pour orienter les campagnes marketing de manière plus ciblée et efficace.

Pour évaluer la performance du modèle, deux indicateurs clés seront privilégiés :

- **L'AUC (Area Under the Curve)**, qui mesure la capacité du modèle à discriminer les assurés intéressés des autres.
- **Le F1-score**, qui équilibre précision et rappel afin de minimiser les faux positifs et faux négatifs.

En maximisant ces deux métriques, le modèle visera à répondre aux objectifs commerciaux tout en respectant les contraintes opérationnelles. Ce rapport décrit la démarche adoptée pour atteindre cet objectif, depuis la préparation des données jusqu'à la sélection du modèle optimal.

Rappels sur les métriques

Les deux métriques utilisées pour évaluer la performance du modèle, le **F1-score** et l'**AUC (Area Under the Curve)**, reposent sur des concepts fondamentaux issus de la **matrice de confusion**. Cette matrice représente les résultats d'une classification binaire et se décompose comme suit :

TP	FN
FP	TN

	Classe Prédite : Positif	Classe Prédite : Négatif
Classe Réelle : Positif	TP (Vrais Positifs)	FN (Faux Négatifs)
Classe Réelle : Négatif	FP (Faux Positifs)	TN (Vrais Négatifs)

F1-score : Le F1-score est calculé à partir de deux métriques dérivées de la matrice de confusion : la précision et le rappel. Ces métriques se définissent comme suit :

- **Précision :** $\text{Précision} = \frac{TP}{TP+FP}$, qui mesure la proportion des prédictions positives qui sont correctes.
- **Rappel :** $\text{Rappel} = \frac{TP}{TP+FN}$, qui mesure la proportion des exemples positifs correctement détectés.

Le F1-score est donné par :

$$F1 = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}.$$

Il est particulièrement utile lorsque les classes sont déséquilibrées, en équilibrant les faux positifs et les faux négatifs.

AUC (Area Under the Curve) : L'AUC est calculée à partir de la courbe ROC (Receiver Operating Characteristic), qui illustre le compromis entre le taux de vrais positifs (TPR) et le taux de faux positifs (FPR) pour différents seuils de classification :

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}.$$

L'aire sous la courbe ROC (AUC) varie entre 0.5 (modèle aléatoire) et 1 (modèle parfait).

Ces outils et métriques sont fondamentaux pour comprendre et interpréter les performances d'un modèle de classification.

Direction du projet

Nous avons beaucoup travaillé sur ce projet, en creusant les pistes qui nous semblaient pertinentes. Nous avons établi un premier modèles en essayant les algorithmes vus en cours et nous sommes rapidement arrivés à un plafonnement des performances du modèle: env. 0.67 en F1-score et env. 0.85 en AUC. Nous avons donc ajouter des méthodes de ré-échantillonnage pour essayer d'améliorer ces métriques.

Nous avons ensuite réfléchi à une seconde méthode, sans ré-échantillonnage.

Nous allons donc présenter ces deux méthodes. Nous avons retenu le premier modèle comme modèle final car c'est lui qui nous donnait les meilleures performances. Cependant, il nous a semblé pertinent de présenter quand même la second méthode car elle nous a demandé beaucoup de travail et que, si nous avions eu plus de temps, nous aurions probablement essayé de prendre les avantages de chacune d'elles pour élaborer un modèle encore plus performant.

Merci d'avance pour votre compréhension et le temps que vous prendrez à lire notre travail.

Table des matières

1	Data Understanding	4
1.1	Exploration et analyse des données (EDA)	4
1.1.1	Focus sur la base de train	4
1.2	Target Variable Analysis	7
1.3	Feature Analysis	8
2	Phase de Pre-Processing	10
2.1	Première méthode	10
2.1.1	Préparation des données	10
2.1.2	Train / Test Set	10
2.1.3	Modification sur les features	11
2.1.4	Pre-Processing final	13
2.2	Seconde méthode	13
2.2.1	Modifications sur les features	13
3	Modélisation	16
3.1	Première méthode	16
3.1.1	Pipeline de pré-traitement	16
3.1.2	Comparaisons des "modèles naifs"	16
3.1.3	Modélisations couplés au méthodes de rééchantillonnage	18
3.1.4	Optimisation des Scores avec la Precision-Recall Curve	21
3.1.5	Choix du modèle final méthode 1	23
3.1.6	Résultats sur les prédictions	24
3.2	Second modèle	24
3.2.1	La régression logistique :	24
3.2.2	Le GradientBoosting Classifier :	25
3.2.3	Le XGBoost Classifier :	26
3.2.4	Le AdaBoostClassifier :	26
3.2.5	Choix du modèle final 2ème méthode	27
3.2.6	Commentaires sur la 2ème méthode	27
4	Conclusion	28
4.1	Modèle retenu	28
4.2	Limites du modèle et optimisations possibles	29
5	Annexe	30

1 Data Understanding

1.1 Exploration et analyse des données (EDA)

1.1.1 Focus sur la base de train

Analyse de la forme:

Variable Target : Response, il s'agit de la variable que l'on cherchera à prédire par la suite;

Lignes et colonnes : 50 000 * 12

Types de variables : 8 int64; 3 objects

Valeurs manquantes : 0

Analyse de fond:

Visualisation de la target : 76 % de réponses favorables / 24 % non favorables; on peut en déduire avec la loi forte des grands nombres qu'en moyenne 24 % des assurés répondront "non" au nouveau produit d'assurance. Cette remarque nous sera d'un très grand intérêt par la suite dans la détermination d'un seuil pour la classification des classes. Par ailleurs, on constate que les classes sont déséquilibrées. En fonction des modèles utilisés, en vue d'avoir une robustesse des coefficients de prédiction (dans une régression par exemple) il peut être intéressant d'explorer des méthodes de rééchantillonnage (**oversampling, undersampling...**). On en reparlera un peu plus loin.

Variables numériques :

- Age
- Annual_Premium
- Vintage

On catégorisera ces variables par la suite en raison du grand nombre de valeurs qu'elles prennent. Et cette catégorisation nous permettra de faire une représentation de toutes les variables afin de voir celles qui sont colinéaires via une analyse en composante multiple (voir plus bas).

Variables catégorielles :

- Response (mise à part car c'est la target)
- Gender
- Driving_License
- Region_Code
- Previously_Insured
- Vehicle_Age
- Vehicle_Damage
- Policy_Sales_Channel

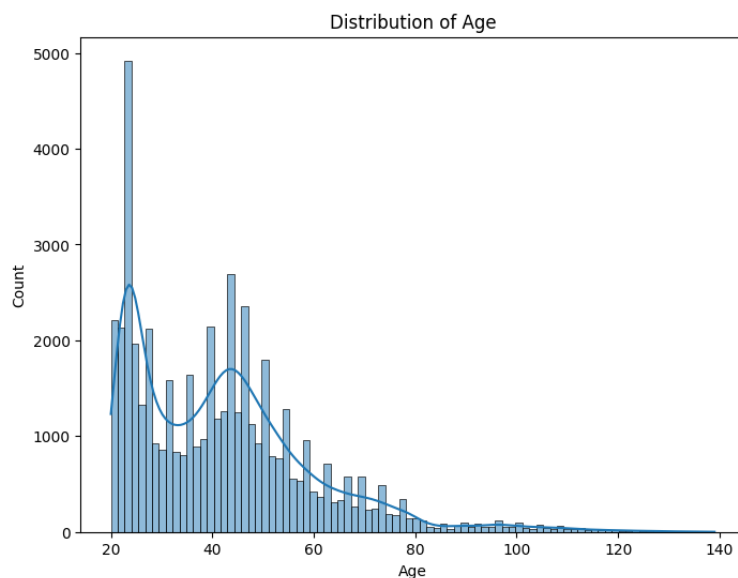


Figure 1: Distribution de l'âge

Lors de l'EDA, nous avons commencé par visualiser les variables afin de voir si certaines présentaient des incohérences ou des éléments particuliers.

Par-exemple, lors de la visualisation des données, on remarque des valeurs abberantes pour l'âge.

Il serait peut-être judicieux de supprimer les observations au-delà d'un certain seuil, par-exemple 115 ans. Dans ce genre d'études, catégoriser la variable *Age* peut être intéressant, c'est ce que nous ferons par la suite.

Nous avons remarqué que la variable *Annual Premium*, bien qu'elle soit continue, ne se concentre qu'autour de 4 valeurs.

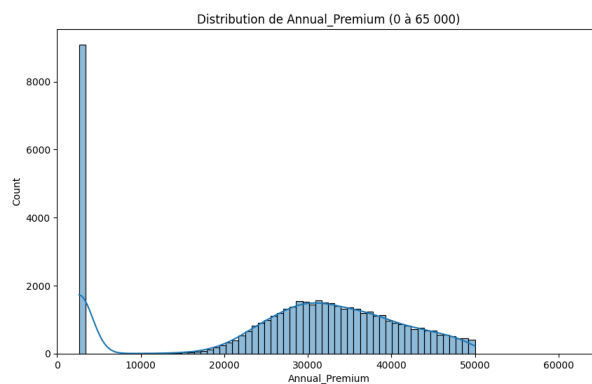


Figure 2: Annual Premium 0

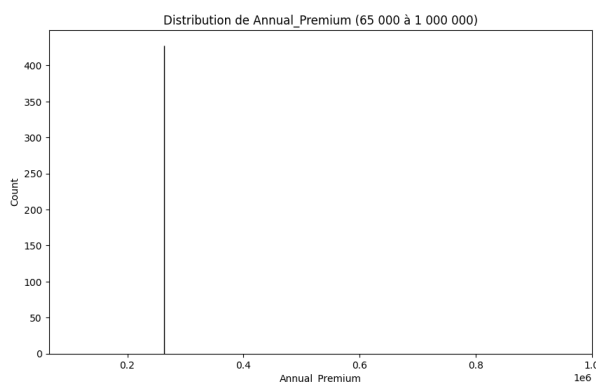


Figure 3: Annual Premium 1

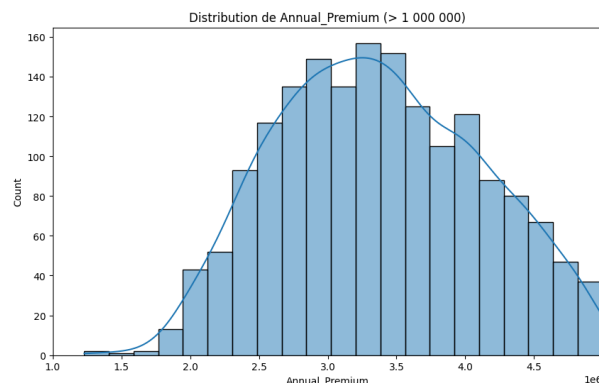


Figure 4: Annual Premium 2

Figure 5: Comparaison des catégories de primes annuelles.

A la lumière de ces représentations, il semblerait qu'il y ait, au moins, 4 groupes distincts pour la variable Annual_Premium:

- Un autour de 4 000.
- Un assez large s'étalant approximativement de 13 000 à 50 000.
- Un petit autour des 200 000.
- Un autre relativement large allant approximativement de 1 000 000 à 5 000 000.

Il serait donc judicieux de catégoriser cette variable.

Pour la variable Vintage, on repère qu'il y a une très grande majorité de "nouveaux assurés" (moins d'un an). Il pourrait être intéressant de faire cette distinction entre "Nouvel assuré" et "Ancien assuré".

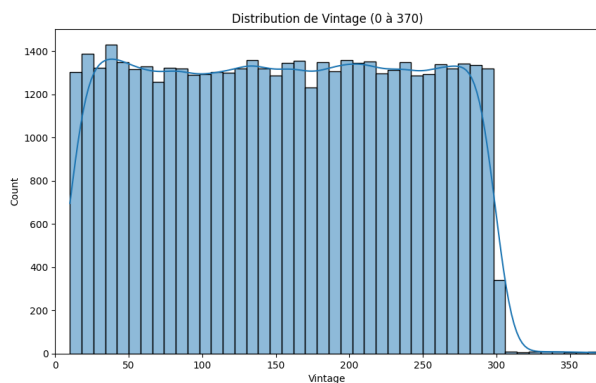


Figure 6: Vintage 1

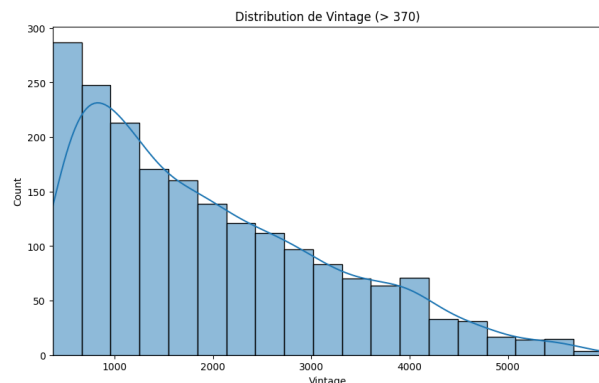


Figure 7: Vintage 2

Figure 8: Comparaison des catégories de primes annuelles.

Nous avons aussi remarqué qu'il y avait beaucoup de modalités pour les variables *Region_Code* et *Policy_Channel* (nous ne mettons pas les graphes ici), dont certaines avec beaucoup d'observations, et d'autres très peu... Il sera donc peut-être judicieux de faire un regroupement de modalités pour ces variables.

1.2 Target Variable Analysis

Nous ne détaillerons ici que certaines des observations que l'on a pu faire lors de l'analyse des données.

Le graphique suivant est très intéressant puisqu'il nous montre une différence dans les distributions des âges en fonction de la réponse de l'assuré. Ceci peut indiquer que la variable *Age* est une bonne variable prédictive importante lors de la modélisation.

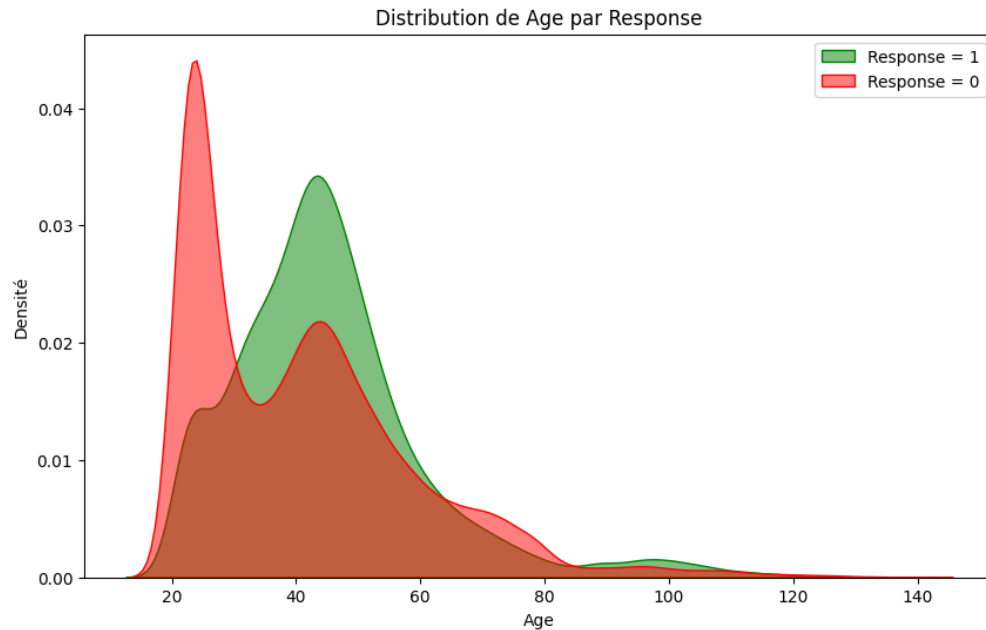
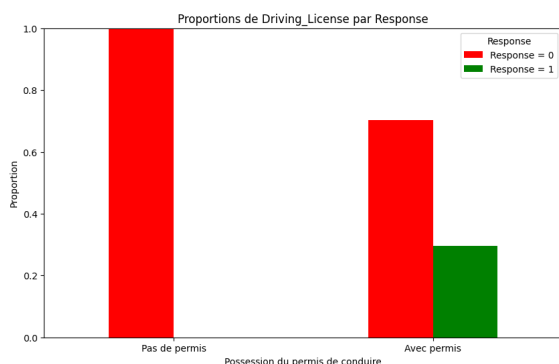
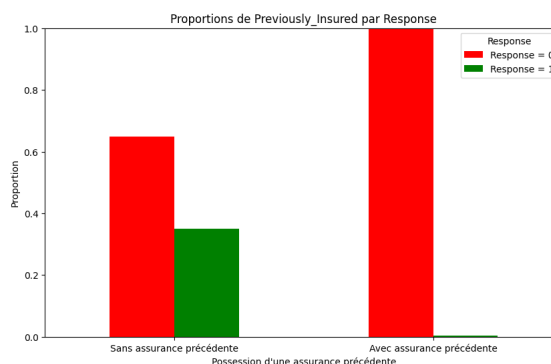


Figure 9: Distribution de Age par Response

Une autre variable importante est, selon nous, la variable *Driving_License*. En effet, tous les individus n'ayant pas le permis ont répondu de manière négative au sondage. On observe le même genre de phénomène pour la variable *Previously_Insured*

Figure 10: *Driving_License* par *Response*Figure 11: *Previously_Insured* par *Response*Figure 12: *Previously_Insured* & *Driving_License* par *Response*

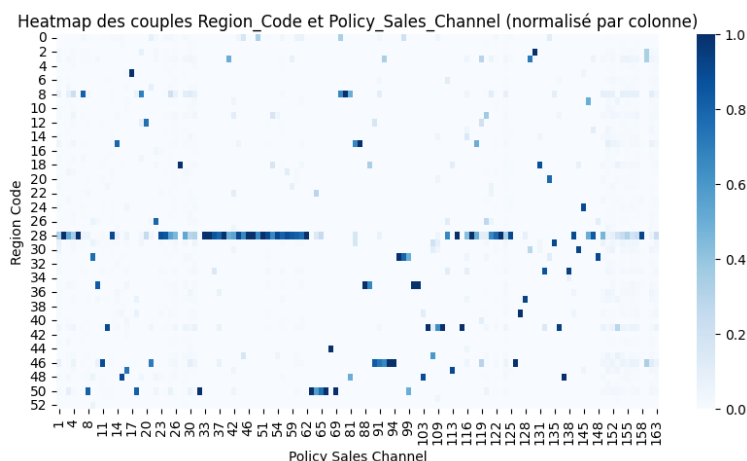
Les 3 graphiques ci-dessus sont intéressants car ils indiquent que les variables **Age**, **Driving License** et **Previously Insured** sont discriminantes. Ces intuitions, tirées de la simple interprétation graphique, ont été confirmées par des tests de Anova ou du khi-2 qui ont établis une relation significative à 95% entre ces variables et la variable **Response**.

Certains de ces résultats nous paraissent assez logiques: un individu qui n'a pas le permis, répondra toujours "NON".

1.3 Feature Analysis

Dans cette partie nous allons étudier quelques relations entre variables explicatives, et essayer de déterminer les plus pertinentes.

Certains couples de *Region_Code* et *Policy_Sales_Channel* semblent être plus présents que d'autres.

Figure 13: Relation entre de *Region_Code* et *Policy_Sales_Channel*

Ci-dessous, la heatmap des taux de réponses pour les couples de ces deux variables :

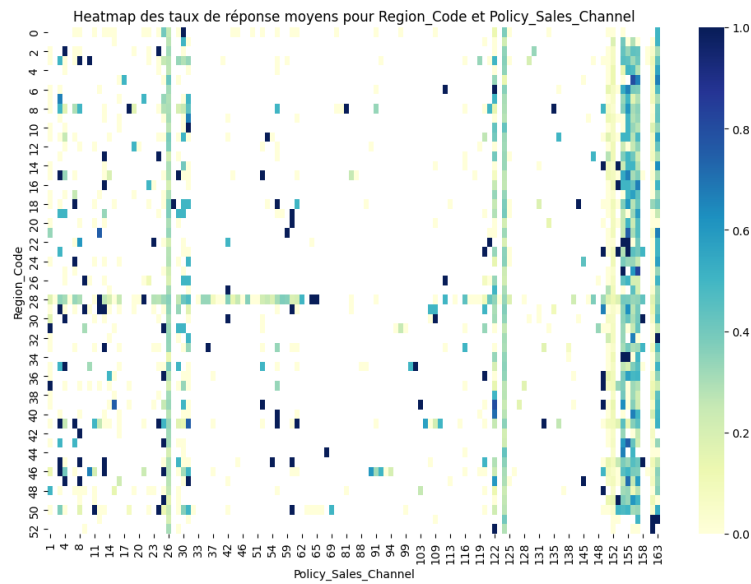


Figure 14: Réponses par-rapport à *Region_Code* et *Policy_Sales_Channel*

Du fait du nombre important de modalités pour ces deux variables, il pourra être intéressant de regrouper certaines modalités ou de réaliser un kmeans sur ces variables.

2 Phase de Pre-Processing

2.1 Première méthode

On peut effectuer plusieurs opérations au cours de l'étape de **pré-processing** des données, notamment :

- Normalisation des données.
- Nettoyage des valeurs aberrantes (*outliers*).
- Gestion des valeurs manquantes (*NaN*).
- Sélection des variables (*Feature Selection*).
- Extraction de nouvelles variables (*Feature Extraction*).
- Encodage des variables catégorielles.

Les objectifs principaux du pré-processing sont les suivants :

1. Mettre les données dans un format propice au Machine Learning.
2. Séparer les ensembles en **Train** et **Test**.
3. Encodage des variables catégorielles.
4. Nettoyage des *NaN* (dans notre cas, nous n'avons pas de valeurs manquantes).

2.1.1 Préparation des données

Pour améliorer la performance du modèle, on pourra envisager :

- Une **sélection des variables** (*Feature Selection*) pour ne conserver que les plus pertinentes.
- De **l'ingénierie des variables** (*Feature Engineering*) en créant des variables supplémentaires.
- Une **mise à l'échelle** des données (*Feature Scaling*).
- La suppression des *outliers* identifiés.

On enlève les variables qu'on a identifiées comme inutile : l'identifiant.

2.1.2 Train / Test Set

Remarque : on fait l'encodage après les modifications des Features pour des raisons pratiques.

Les données ont été divisées en deux ensembles : 80% pour l'entraînement et 20% pour le test. La répartition des classes pour la variable cible (**Response**) montre un déséquilibre notable :

- Environ 75% des observations correspondent à la classe 0 (non intéressé).
- Environ 25% des observations correspondent à la classe 1 (intéressé).

```
1 from sklearn.model_selection import train_test_split
2
3 train, test = train_test_split(df, test_size = 0.2, random_state = 42)
4
5 print(train['Response'].value_counts(normalize = True))
6 print(test['Response'].value_counts(normalize = True))
```

```
1 Response
2 0    0.75665
3 1    0.24335
4 Name: proportion, dtype: float64
5 Response
6 0    0.7533
7 1    0.2467
8 Name: proportion, dtype: float64
```

Les classes étant déséquilibrées (en faveur de ceux qui ne sont pas intéressés), il faudra penser à utiliser des techniques de rééchantillonnage.

2.1.3 Modification sur les features

Pour la variable **Vintage**, initialement exprimée en jours, une conversion en années a été effectuée, suivie d'une catégorisation en deux groupes : **< 0.87 years** et **> 0.87 years** (Assurés fidèles Nouveaux assurés). Cette transformation simplifie l'interprétation tout en conservant l'information pertinente.

La variable **Annual Premium**, représentant le montant de la prime annuelle, a été catégorisée en quatre classes (**Low, Medium, High, Very High**) pour réduire l'impact des valeurs extrêmes. Une transformation logarithmique a également été ajoutée afin de normaliser la distribution et améliorer la robustesse des modèles.

Pour la variable **Age**, les valeurs aberrantes ont été abaissées à 110 et les données ont été regroupées en intervalles (**18-27, 28-37, etc.**). Ces intervalles ont été définis pour capturer les tendances observées dans la relation entre l'âge et la variable cible **Response**.

Les variables catégorielles ayant de nombreuses modalités, comme **Policy_Sales_Channel** et **Region.Code**, ont été simplifiées en regroupant les catégories peu représentées dans une catégorie générique (**autre**). Ces regroupements ont été réalisés avec des seuils fixés à 50 et 250 occurrences respectivement.

Pour réduire la complexité des données tout en capturant des relations non linéaires entre certaines variables, un clustering K-means a été appliqué. Ce clustering a été effectué sur les variables **Region.Code**, **Policy_Sales_Channel**, **Annual Premium**.

Encodage des variables catégorielles L'encodage est une étape essentielle pour rendre les données exploitables par les modèles de machine learning.

Dans cette partie, plusieurs stratégies d'encodage ont été appliquées, en fonction du type de variable et de leur relation potentielle avec la variable cible **Response**. Voici les principales étapes réalisées :

Pour les variables catégorielles **Vehicle_Damage**, **Gender** et **Vehicle_Age**, un encodage simple basé sur un dictionnaire de correspondance a été utilisé :

- **Vehicle_Damage** : transformée en 1 (Yes) ou 0 (No).
- **Gender** : transformée en 1 (Male) ou 0 (Female).
- **Vehicle_Age** : convertie en une variable ordinale, avec les valeurs 0 pour < 1 Year, 1 pour 1-2 Year et 2 pour > 2 Years, en supposant un ordre logique lié à l'ancienneté du véhicule.

Pour les variables catégorielles issues des transformations, telles que **Age_Cat**, **Annual_Premium_Category**, **Cluster** (résultat du K-means) et **Vintage_Cat**, un **encodage One-Hot** a été appliqué. Cette méthode consiste à créer une nouvelle colonne pour chaque modalité, avec une valeur 1 si l'observation appartient à cette modalité, et 0 sinon. Cela permet de représenter chaque catégorie sans imposer d'ordre arbitraire entre elles.

Dans nos modèles, nous avons choisi de ne pas inclure les variables **Policy_Sales_Channel** et **Region_Code**. Ces variables présentent un très grand nombre de modalités, ce qui peut complexifier inutilement le traitement des données et allonger considérablement le temps d'entraînement des modèles.

Bien que cette décision puisse être discutée, nous avons estimé qu'elle permettrait de gagner du temps tout en limitant le risque de surapprentissage lié à une mauvaise gestion de ces variables. Cela reste cependant un compromis qui pourrait être amélioré dans des itérations futures, notamment en explorant des techniques plus adaptées pour gérer les variables à haute cardinalité.

```
1 def encode_columns(df):
2     code_vehicle_damage = {'Yes': 1, 'No': 0}
3     code_gender = {'Male': 1, 'Female': 0}
4     code_vehicle_age = {'< 1 Year': 0, '1-2 Year': 1, '> 2 Years': 2}
5
6     df['Vehicle_Damage'] = df['Vehicle_Damage'].map(code_vehicle_damage)
7     df['Gender'] = df['Gender'].map(code_gender)
8     df['Vehicle_Age'] = df['Vehicle_Age'].map(code_vehicle_age)
9
10    # One-Hot Encoding
11    df = pd.get_dummies(df, columns=['Age_Cat', 'Annual_Premium_Category', 'Cluster',
12    'Vintage_Cat'], drop_first=True)
13
14    return df
```

2.1.4 Pre-Processing final

La fonction de prétraitement final a pour objectif de transformer les données brutes en un format adapté à notre modèle de machine learning. Elle comprend plusieurs étapes clés :

- **Conversion et catégorisation de Vintage** : la colonne est convertie en années, puis regroupée en deux catégories : < 0.87 years et > 0.87 years.
- **Catégorisation des variables** :
 - Age : regroupé en intervalles comme 18-27, 28-37, etc.
 - Annual.Premium : catégorisé en niveaux (Low, Medium, etc.) et transformé logarithmiquement pour lisser les valeurs.
 - Policy_Sales_Channel et Region_Code : regroupés en fonction de leur fréquence.
- **Clustering K-Means** : un algorithme de clustering est appliqué sur plusieurs variables pour créer des segments pertinents et surtout pour réduire la cardinalité des variables
- **Encodage des variables catégoriques** : les variables catégoriques sont transformées en variables indicatrices via un encodage One-Hot.

Les colonnes redondantes ou inutiles (id, Age, Vintage, etc.) sont ensuite supprimées pour simplifier les données. La fonction complète est présentée ci-dessous :

```
1 def preprocessing(df):
2     df = categorize_vintage(df) # met en annee et categorise "> 87" ou "> 87"
3     df = categorize_age(df) # categorise l'age
4     df = categorize_annual_premium(df) #categorise la prime annuelle
5     df = transform_log_annual_premium(df) # transformation logarithmique de la
6         prime
7     df = categorize_int_policy_sales_channel(df, threshold=25) # categorise les
8         canaux de distribution
9     df = categorize_int_region_code(df, threshold=250) # categorise les regions
10    df = kmeans_region_policy_premium(df, n_clusters=10) # on fait un kmeans pour
11        les regions, les canaux, et prime
12    df = encode_columns(df) # encodage
13
14    df.drop(['id', 'Age', 'Vintage', 'Annual_Premium', 'Region_Code', '
15        Policy_Sales_Channel'], axis=1, inplace=True, errors='ignore')
16    return df
```

2.2 Seconde méthode

2.2.1 Modifications sur les features

Pour cette méthode, nous nous sommes concentrés sur les variables catégorielles et leur catégorisation.

Pour la variable Region_code, nous avons d'abord trié les régions en fonction du nombre de réponses positives par ordre décroissant. Nous avons ensuite regrouper les régions en 3 catégories: les 5 régions avec le plus de réponses positives, les 5 suivantes et le reste des régions.

```
1 sorted_regions = region_response_count.sort_values(ascending=False)
2
3 top_5_regions = sorted_regions.index[:5] # Les 5 regions avec le plus de r ponses
   positives
4 next_5_regions = sorted_regions.index[5:10] # Les 5 suivantes
5 remaining_regions = sorted_regions.index[10:] # Le reste des regions
6
7 # Fonction d'assignation de groupe de r gion
8 def assign_region_group(region):
9     if region in top_5_regions:
10         return 1 # Groupe 1 : Les 5 premieres r gions
11     elif region in next_5_regions:
12         return 2 # Groupe 2 : Les 5 suivantes
13     else:
14         return 3 # Groupe 3 : Toutes les autres regions
15
16 # Appliquer la fonction pour cr er la colonne de regroupement des regions dans df1
17 df1['Region_Group'] = df1['Region_Code'].apply(assign_region_group)
```

Pour la variable **Vintage**, initialement exprimée en jours, une conversion en années a été effectuée, suivie d'une catégorisation en 5 groupes : $\text{vintage} < 0.25$ years; $0.25 \leq \text{vintage} < 0.5$; $0.5 \leq \text{vintage} < 0.75$; $0.75 \leq \text{vintage} < 1$; et $\text{vintage} > 1$. Cette transformation simplifie l'interprétation tout en conservant l'information pertinente.

Concernant la variable **Age**, les valeurs aberrantes n'ont pas été supprimées. Les données ont été regroupées en intervalles définis par rapport à ce qui nous semblait être des étapes de la vie qui correspondent à une utilisation de la voiture précise. Bien sûr, cela reste un modèle et souffre donc des éléments de généralité.

```
1 def categorize_age(age):
2     if 18 <= age < 30: # Jeune conducteur
3         return 1
4     elif 30 <= age < 40: # Probablement creation de famille
5         return 2
6     elif 40 <= age < 50: # Amelioration et stabilisation de la situation
   conomique
7         return 3
8     elif 50 <= age < 60: # Senior (peut-etre moins de moyen de transport personnel)
9         return 4
10    else:
   #Senior +
11    return 5
```

La variable **Annual_Premium** a fait l'objet du même featuring que pour la première méthode, à savoir: catégorisation en quatre classes (Low, Medium, High, Very High) et transformation logarithmique.

La variable **Policy_Sales_Channel** a été traitée de manière assez similaire à **Region_code**: une catégorisation selon le nombre de réponses positives.

```
1 policy_response_count = df1[df1['Response'] == 1].groupby('Policy_Sales_Channel').
   size()
2
3 #Trier les canaux par le nombre de r ponses positives
```

```

4 sorted_channels = policy_response_count.sort_values(ascending=False)
5
6 # Cat goriser les canaux en trois groupes
7 top_2_channels = sorted_channels.index[:2]
8 next_6_channels = sorted_channels.index[2:8]
9
10 # Fonction de regroupement des canaux
11 def assign_channel_group(channel):
12     if channel in top_2_channels:
13         return 1 # Groupe 1 : les deux canaux avec le plus de reponses positives
14     elif channel in next_6_channels:
15         return 2 # Groupe 2 : les six suivants
16     else:
17         return 3 # Groupe 3 : le reste

```

Pour les variables catégorielles **Vehicle_Damage**, **Gender** et **Vehicle_Age**, un encodage simple basé sur un dictionnaire de correspondance a été utilisé :

- **Vehicle_Damage** : transformée en 1 (Yes) ou 0 (No).
- **Gender** : transformée en 1 (Male) ou 0 (Female).
- **Vehicle_Age** : convertie en une variable ordinale, avec les valeurs 0 pour < 1 Year, 1 pour 1-2 Year et 2 pour > 2 Years, en supposant un ordre logique lié à l'ancienneté du véhicule.

A l'issue de toutes ces modifications, notre base de données est constituée des variables suivantes:

Driving_License_0
Age_1
Age_2
Age_3
Age_4
Vehicle_Damage_1
Channel_Group_2
Previously_Insured_0
Region_Group_2
Region_Group_1
Gender_0
Vintage_1
Vintage_2
Vintage_3
Vehicle_Age_2
Channel_Group_3
Age_2
Age_3
Age_4
Annual_Premium_very_high
Annual_Premium_Low

Table 1: Liste des titres de colonnes

3 Modélisation

3.1 Première méthode

3.1.1 Pipeline de pré-traitement

Un pipeline de prétraitement a été défini pour standardiser et sélectionner les meilleures caractéristiques :

- Réduction de la dimension avec `PCA()` afin de garder 97,5 % de l'information.
- Une normalisation des données avec `StandardScaler()`
- Transformation polynomiale de degré 2 des variables.
- Sélection des caractéristiques les plus pertinentes avec `SelectKBest`.

```

1 preprocessor = make_pipeline(
2     PCA(n_components=0.975),
3     StandardScaler(),
4     PolynomialFeatures(degree=2, include_bias=False),
5     SelectKBest(f_classif, k=10)
6 )

```

3.1.2 Comparaisons des "modèles naifs"

On parle ici de modèles naifs car nous n'avons pas appliqué les techniques de rééchantillonnage dans cette première version.

Nous avons choisi de comparer les modèles suivants :

```

1 DecisionTree = make_pipeline(preprocessor, DecisionTreeClassifier(random_state =
2     42))
3 RandomForest = make_pipeline(preprocessor, RandomForestClassifier(random_state =
4     42))
5 AdaBoost = make_pipeline(preprocessor, AdaBoostClassifier(random_state = 42))
6 GradientBoosting = make_pipeline(preprocessor, GradientBoostingClassifier(
7     random_state = 42))
8 XGBoost = make_pipeline(preprocessor, XGBClassifier(random_state = 42))

```

Table 2: Résumé des performances des modèles sans techniques de rééchantillonnage

Modèle	F1-score	AUC	Précision (classe 1)	Recall (classe 1)	Accuracy
DecisionTree	0.47	0.65	0.48	0.46	0.74
RandomForest	0.57	0.72	0.57	0.57	0.79
AdaBoost	0.60	0.74	0.59	0.62	0.80
GradientBoosting	0.62	0.75	0.59	0.65	0.80
XGBoost	0.62	0.75	0.58	0.67	0.80

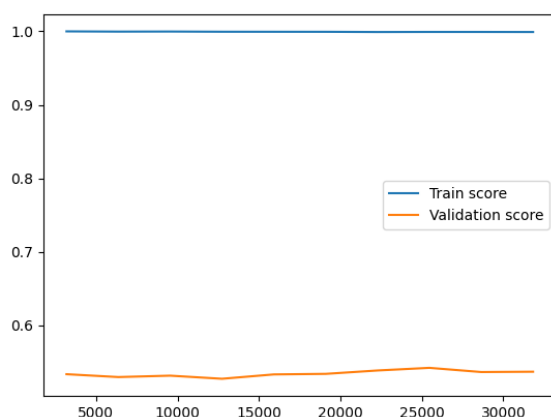


Figure 15: Learning Curve de DecisionTreeClassifier

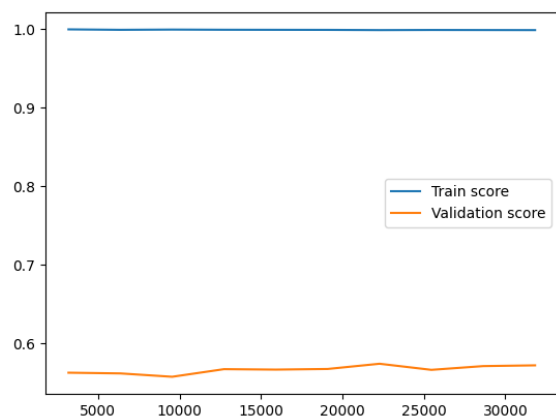


Figure 16: Learning Curve de RandomForestClassifier

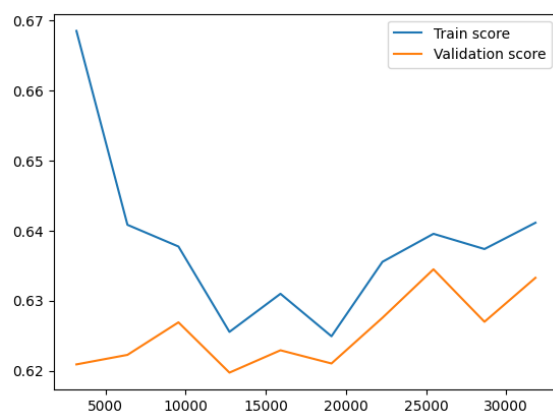


Figure 17: Learning Curve de AdaBoostClassifier

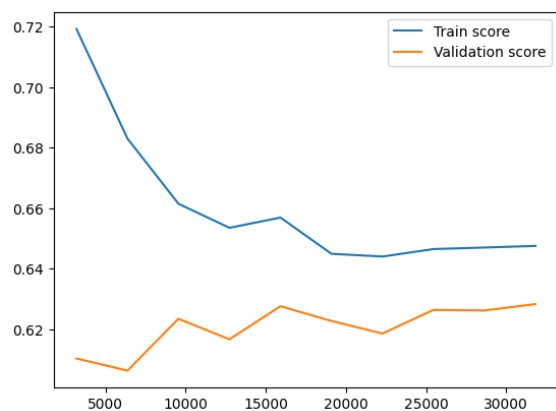


Figure 18: Learning Curve de GradientBoostingClassifier

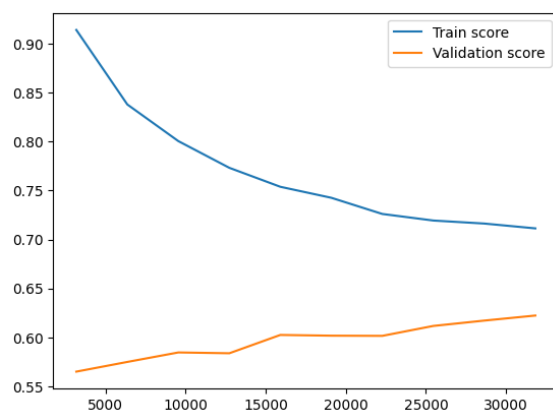


Figure 19: Learning Curve de XGBoostClassifier

Bilan des performances des modèles

Les courbes d'apprentissage et les métriques issues des différents modèles permettent de dégager les observations suivantes :

- **Modèles performants :**

- **GradientBoosting** et **XGBoost** se distinguent par des **F1-scores élevés** (0.61 et 0.62 respectivement) et des **AUC** atteignant 0.75 et 0.76.

Les **courbes d'apprentissage** montrent une bonne stabilité entre les ensembles d'entraînement et de validation, traduisant une **bonne capacité de généralisation**.

- **AdaBoost** reste compétitif avec un **F1-score** de 0.60 et un **AUC** de 0.74, bien qu'il soit légèrement en dessous des deux premiers modèles.

- **Modèles moins performants :**

- **DecisionTree** affiche un **F1-score faible** (0.47) et un **AUC** de 0.65, avec des courbes indiquant un **fort sur-apprentissage**.
- **RandomForest**, bien qu'améliorant les résultats du DecisionTree avec un **F1-score** de 0.57 et un **AUC** de 0.71, présente encore des écarts notables entre les scores d'entraînement et de validation, suggérant un potentiel d'optimisation.

Conclusion : Les modèles **GradientBoosting** et **XGBoost** sont les **plus prometteurs** grâce à leur équilibre entre biais et variance. À l'inverse, le **DecisionTree** est peu performant, tandis que **RandomForest** et **AdaBoost** présentent un potentiel d'amélioration.

Aucun **rééchantillonnage** des données (par exemple SMOTE) ou **optimisation des hyperparamètres** (RandomizedSearchCV ou GridSearchCV) n'a encore été appliqué, ce qui laisse une marge de progression pour améliorer les performances globales des modèles.

3.1.3 Modélisations couplés aux méthodes de rééchantillonnage

Comme mentionné précédemment, un problème de **déséquilibre de classes** existe dans nos données. Ce déséquilibre peut être corrigé en appliquant diverses méthodes de rééchantillonnage. Voici une description des techniques décrites dans le notebook :

1. **SMOTE (Synthetic Minority Over-sampling Technique) :**

- SMOTE est une méthode de **sur-échantillonnage** qui génère des exemples synthétiques pour la classe minoritaire. Contrairement au rééchantillonnage classique, où des copies exactes d'exemples sont créées, SMOTE génère de nouvelles instances en **interpolant** entre des exemples existants de la classe minoritaire.
- Cette technique aide à équilibrer les classes sans créer de doublons, ce qui rend le modèle plus robuste face au déséquilibre.

2. **Random Under Sampling :**

- Le sous-échantillonnage aléatoire consiste à **réduire la taille** de la classe majoritaire en sélectionnant un sous-ensemble aléatoire de ses exemples.
- Bien qu'il aide à rétablir l'équilibre entre les classes, cette méthode comporte un risque de **perdre des informations** importantes contenues dans la classe majoritaire.

3. SMOTENC (SMOTE for Nominal and Continuous features) :

- SMOTENC est une variante de SMOTE spécialement conçue pour traiter des données comportant à la fois des variables **catégorielles et continues**.
- Elle utilise des mécanismes spécifiques pour gérer les variables catégorielles et génère des exemples synthétiques tout en respectant les catégories.
- Il est nécessaire de spécifier les variables catégorielles lors de l'application de cette méthode.

4. SMOTE avec Tomek Links :

- Cette méthode combine SMOTE avec l'élimination des **liens de Tomek**. Les liens de Tomek sont des paires d'exemples très proches dans l'espace des caractéristiques, mais appartenant à des classes différentes.
- En supprimant ces liens, on améliore la **ségrégation des classes** et on nettoie les frontières entre elles après le sur-échantillonnage.

5. Random Over Sampling :

- Le sur-échantillonnage aléatoire consiste à **répliquer** des exemples de la classe minoritaire pour augmenter sa taille. Contrairement à SMOTE, cette méthode ne crée pas de nouveaux exemples synthétiques, mais copie des exemples existants.
- Bien que simple à implémenter, cette technique présente un risque de **sur-ajustement** si les exemples répliqués ne sont pas suffisamment variés.

Conclusion : L'application de ces méthodes de rééchantillonnage permet de traiter le déséquilibre de classes, ce qui peut améliorer les performances des modèles. Le choix de la méthode dépend du type de données et des objectifs du projet, chaque approche ayant ses avantages et ses inconvénients.

Nous avons décidé de comparer les 4 techniques évoqués dans le code ci-dessous :

```
1 resamplers = {'SMOTE': SMOTE(random_state=42),
2               'RandomUnderSampler': RandomUnderSampler(random_state=42),
3               'RandomOverSampler': RandomOverSampler(random_state=42),
4               'SMOTETomek': SMOTETomek(random_state=42)
5             }

6
7
8
9
10
11
12
13
14
15
16
17 results = []
for name, model in dict_of_models.items():
    for resampler_name, resampler in resamplers.items():
        # R échantillonner les données
        X_resampled, y_resampled = resampler.fit_resample(X_train, y_train)

        # Utilisation de pd.isna() pour détecter les valeurs manquantes
        mask = X_resampled.isna().any(axis=1)

        # Suppression des lignes avec des valeurs manquantes
        X_resampled = X_resampled[~mask]
        y_resampled = y_resampled[~mask]

        # Entraîner le modèle avec les données rééchantillonnées
        model.fit(X_resampled, y_resampled)

        # Prédire les étiquettes sur le set de test
```

```

18     y_pred = model.predict(X_test)
19
20     # Calculer les m triques
21     f1 = f1_score(y_test, y_pred)
22     auc = roc_auc_score(y_test, model.predict_proba(X_test)[: , 1]) # AUC avec
        predict_proba
23
24     # Stocker les r sultats
25     results.append({
26         'Model': name,
27         'Resampling Technique': resampler_name,
28         'F1 Score': f1,
29         'AUC': auc,
30         'Accuracy': model.score(X_test, y_test),
31         'Precision': precision_score(y_test, y_pred),
32         'Recall': recall_score(y_test, y_pred)
33     })
34
35     # Afficher les r sultats pour chaque combinaison de modele et technique de
        reechantillonnage
36     print(f'Model: {name}, Resampler: {resampler_name}, F1: {f1:.4f}, AUC: {auc
        :.4f}')
37
38 # Cr er un DataFrame pour analyser les r sultats
39 combined_model_resampler = pd.DataFrame(results)

```

Model	Resampling Technique	F1 Score	AUC	Accuracy	Precision	Recall
DecisionTree	SMOTE	0.54	0.70	0.76	0.51	0.57
DecisionTree	RandomUnderSampler	0.56	0.72	0.76	0.51	0.63
DecisionTree	RandomOverSampler	0.44	0.64	0.76	0.51	0.39
DecisionTree	SMOTETomek	0.47	0.65	0.76	0.52	0.43
RandomForest	SMOTE	0.61	0.86	0.79	0.56	0.68
RandomForest	RandomUnderSampler	0.66	0.86	0.78	0.54	0.84
RandomForest	RandomOverSampler	0.52	0.87	0.79	0.59	0.46
RandomForest	SMOTETomek	0.62	0.87	0.79	0.57	0.67
AdaBoost	SMOTE	0.68	0.88	0.78	0.53	0.94
AdaBoost	RandomUnderSampler	0.67	0.87	0.77	0.51	0.97
AdaBoost	RandomOverSampler	0.67	0.88	0.77	0.52	0.97
AdaBoost	SMOTETomek	0.68	0.88	0.78	0.53	0.94
GradientBoosting	SMOTE	0.68	0.88	0.78	0.53	0.95
GradientBoosting	RandomUnderSampler	0.67	0.88	0.77	0.52	0.96
GradientBoosting	RandomOverSampler	0.68	0.88	0.78	0.54	0.94
GradientBoosting	SMOTETomek	0.68	0.88	0.78	0.53	0.95
XGBoost	SMOTE	0.68	0.87	0.79	0.55	0.88
XGBoost	RandomUnderSampler	0.67	0.87	0.78	0.54	0.89
XGBoost	RandomOverSampler	0.66	0.87	0.79	0.55	0.84
XGBoost	SMOTETomek	0.66	0.87	0.79	0.54	0.85

Table 3: Performance des modèles avec différentes techniques de rééchantillonnage

- Globalement, les techniques de rééchantillonnage ont amélioré les scores des modèles
- **AdaBoost** avec SMOTE, RandomUnderSampler et SMOTETomek se distingue comme l'un des meilleurs modèles, avec un F1-score élevé et un rappel de 0.94, montrant sa capacité à bien identifier les classes minoritaires.
- Les modèles **GradientBoosting** et **XGBoost** ont également montré des performances solides, notamment avec des F1-scores de 0.68 et des rappels supérieurs à 0.94 dans la plupart des configurations de rééchantillonnage.
- **RandomForest** a obtenu de bons résultats, en particulier avec SMOTE et SMOTETomek, où il atteint des F1-scores autour de 0.62, mais ne surpasse pas les performances d'AdaBoost et de GradientBoosting.
- **DecisionTree** a globalement des résultats inférieurs, avec des F1-scores autour de 0.44 à 0.56, ce qui montre une difficulté à gérer le déséquilibre de classes, même avec des techniques de rééchantillonnage.
- Les techniques de rééchantillonnage comme **SMOTE** et **SMOTETomek** semblent améliorer de manière significative les performances des modèles, surtout pour les classes minoritaires, comparées au RandomOverSampler.
- En général, les **méthodes de sur-échantillonnage** et de **nettoyage des frontières** (comme SMOTE-Tomek) ont permis d'améliorer la séparation entre les classes et de renforcer la capacité de détection des classes minoritaires, en particulier dans AdaBoost, GradientBoosting et XGBoost.

3.1.4 Optimisation des Scores avec la Precision-Recall Curve

Afin d'optimiser les performances de nos modèles, notamment dans un contexte de déséquilibre des classes, nous avons exploré l'utilisation des **Precision-Recall Curves**. Cette courbe permet d'analyser la performance d'un modèle en fonction de son seuil de classification, en illustrant la relation entre la **précision** et le **rappel** pour différentes valeurs de seuil.

Nous avons varié le seuil de décision des modèles et observé l'impact sur la précision et le rappel. Les **Precision-Recall Curves** nous ont permis de trouver un bon compromis entre ces deux métriques, ce qui est particulièrement utile dans le cas des classes déséquilibrées, où une faible précision peut souvent être acceptable pour améliorer le rappel, ou vice versa.

Malgré les efforts pour ajuster le seuil de manière optimale, les recherches effectuées via **RandomizedSearchCV** n'ont pas permis de trouver une amélioration significative, bien que cela ait contribué à mieux comprendre les limites des modèles dans ce contexte particulier.

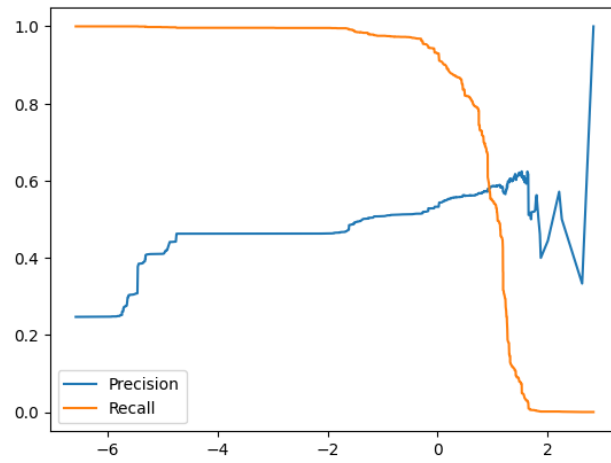


Figure 20: Precision Recall Curve pour le GradientBoosting avec RandomUnderSampling

3.1.5 Choix du modèle final méthode 1

Le modèle retenu pour cette méthode, un **Gradient Boosting Classifier**, s'est avéré être le plus performant parmi les approches testées. Il combine à la fois la puissance des arbres de décision et la capacité d'optimisation via le boosting, offrant ainsi une modélisation robuste adaptée à nos données.

Après un processus rigoureux d'entraînement, d'optimisation des hyperparamètres et d'évaluation, ce modèle a démontré sa capacité à équilibrer efficacement les métriques de performance, notamment :

- Un **F1-Score** optimal de **0.68**, maximisé grâce à un ajustement du seuil de décision (**0.60**), garantissant un compromis judicieux entre précision et rappel.
- Une **AUC (Area Under the Curve)** de **0.83**, confirmant une bonne capacité de discrimination entre les classes.
- Une précision accrue pour la classe dominante tout en minimisant les erreurs sur la classe minoritaire.

L'ajustement des paramètres clés, tels que la profondeur maximale des arbres, le taux d'apprentissage, et le nombre d'itérations, a permis d'éviter le sur-apprentissage, tout en tirant parti des informations complexes contenues dans les données.

De plus, l'utilisation d'un pipeline intégré avec des techniques avancées comme le **PCA** (Analyse en Composantes Principales) pour la réduction de la dimensionnalité, et la normalisation des données a contribué à améliorer la stabilité et la performance globale du modèle.

Table 4: Résumé des paramètres du modèle, pipeline et rééchantillonnage

Composant	Description et Paramètres
Modèle	Gradient Boosting Classifier (<code>n_estimators=50</code> , <code>max_depth=5</code> , <code>learning_rate=0.2</code>)
Pipeline	• PCA (<code>n_components=0.975</code>) : réduction de dimensionnalité • StandardScaler : normalisation des variables • PolynomialFeatures (<code>degree=2</code>) : génération de nouvelles variables polynomiales • SelectKBest (<code>k=15</code>) : sélection des 15 meilleures variables explicatives
Rééchantillonnage	Random UnderSampling (<code>sampling_strategy=0.9</code>) : équilibrage des classes en sous-échantillonnant la classe majoritaire

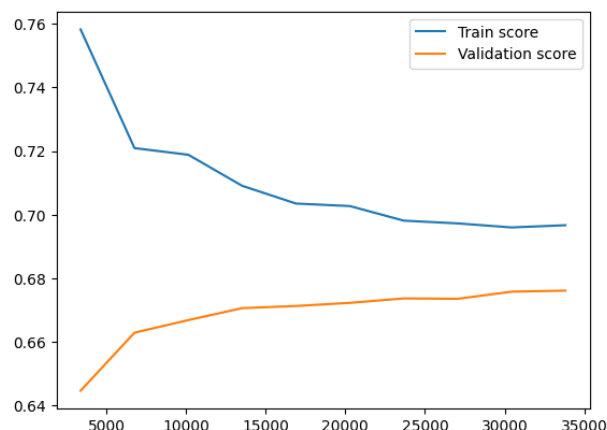


Figure 21: Learning Curve du modèle final

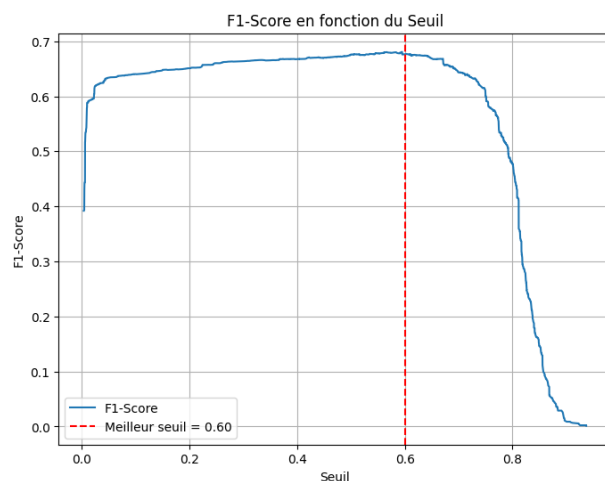


Figure 22: F1-Score en fonction du seuil pour le modèle final

3.1.6 Résultats sur les prédictions

Notre modèle final tel qu'il est défini au-dessus ne nous renvoie que très peu de réponses favorables (moins de 1%...). Afin qu'il corresponde mieux à ce qu'on s'attend, c'est-à-dire à environ 75% de réponses négatives, on a décidé de "tricher" un peu et de baisser le **seuil de prédiction à 0.25**.

3.2 Second modèle

Pour cette seconde méthode, les modèles testés sont : Régression logistique, GradientBoosting classifier, XGBoost Classifier, et le Ada Boost Classifier.

La comparaison des modèles utilisés dans cette méthode est présentée dans un tableau à la fin.

3.2.1 La régression logistique :

Pour cette modélisation, nous avons fait appel à la commande (dont le solver permet de gérer le cas des échantillons en faible proportion) :

$$model = LogisticRegression(solver='lbfgs')$$

Par ailleurs, dans le choix du seuil (de la matrice de confusion, par défaut à 0.5), nous nous sommes inspirés de la Loi Forte des Grands Nombres. En effet, nous avons évoqué plus haut que 24 % des individus répondaient favorablement au produit d'assurance. Dans notre base de *train*, nous avons 50000 observations, ce qui est relativement grand pour dire que la probabilité de répondre "oui" est autour de 0,24 modulo les erreurs d'estimations. De ce fait, nous avons choisi un seuil assez proche de cette probabilité. Nous n'avons pas pris exactement 0,24 comme seuil sinon on aurait eu une très bonne prédiction de la classe **1**, mais pas de la classe **0**.

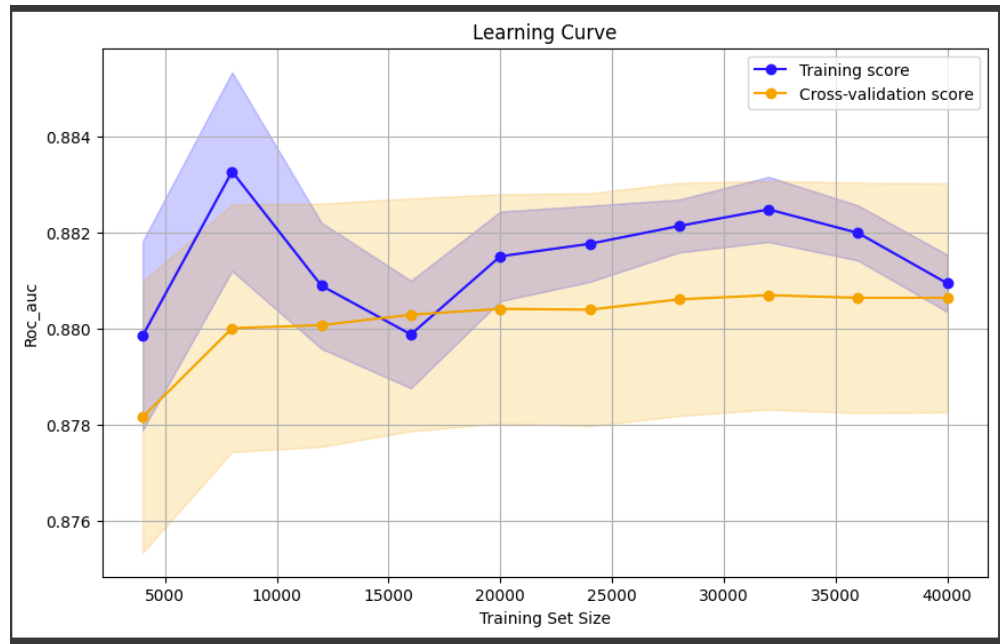
Nous voulions également éviter l'erreur du surapprentissage. Lors des premiers tests avec 0,24 comme seuil, la prédiction de la classe **1** était correcte dans plus de 95% des cas mais celle de la classe **0** était autour de 70%. Nous avons donc privilégié un seuil qui permettait d'avoir un équilibre. Après plusieurs tests, nous

avons finalement retenu comme seuil (qui se trouve dans l'intervalle de confiance d'estimation de la moyenne à partir de la moyenne empirique) :

$$0.365$$

Ce choix est venu se confirmer par un code qui nous a permis de prendre le meilleur seuil qui améliorait le **F1-score** et ce seuil était autour de 0,34. Mais nous avons maintenu 0.365 car il donnait un score à peu près égal à celui obtenu avec 0,34 et avait une bonne **AUC**.

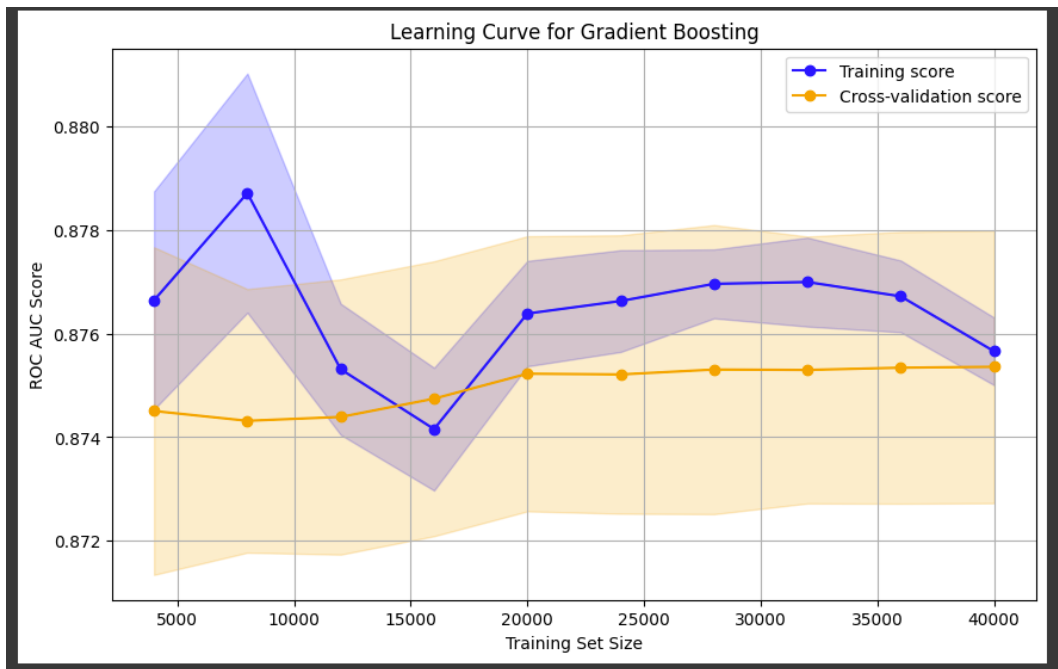
Ci-dessous la courbe de l'AUC avec le seuil retenu.



3.2.2 Le GradientBoosting Classifier :

Tout comme pour la régression logistique, nous prenons un seuil qui est autour de la probabilité empirique d'être dans la classe **1**, c'est-à-dire de répondre positivement à la proposition du nouveau contrat d'assurance. Mais ici, le seuil diffère légèrement de celui utilisé plus haut. Nous avons choisi ici : 0,32. En effet, il nous permettait d'avoir de meilleurs scores que ceux obtenus avec 0,365.

Ci-dessous la courbe de l'AUC avec le seuil retenu.



3.2.3 Le XGBoost Classifier :

Nous prenons un seuil ajusté, à peu près égal au deux précédents : 0,39 qui nous permet d'avoir de meilleurs scores que ceux obtenus avec les deux seuils précédents.

3.2.4 Le AdaBoostClassifier :

Le AdaBoost quant à lui, s'est très bien adapté aux données et a pu constater le déséquilibre entre elles. Nous n'avons pas eu besoin d'ajuster le seuil à un niveau très proche de la probabilité de répondre favorablement à la proposition de souscription au nouveau produit d'assurance. En effet, avec les trois seuils précédents, le taux de prédiction de la classe **1** était au dessus de 90% et souvent même proche de 95%. Quant à la classe **0**, nous étions à 70% à peu près. Dans un souci d'équilibre, nous avons ajusté le seuil à un niveau proche de 0,5. Mais nous avons retenu 0,49 comme seuil dans un souci d'optimisation du F1 score et de l'AUC.

Ci-dessous la courbe de l'AUC avec le seuil retenu.

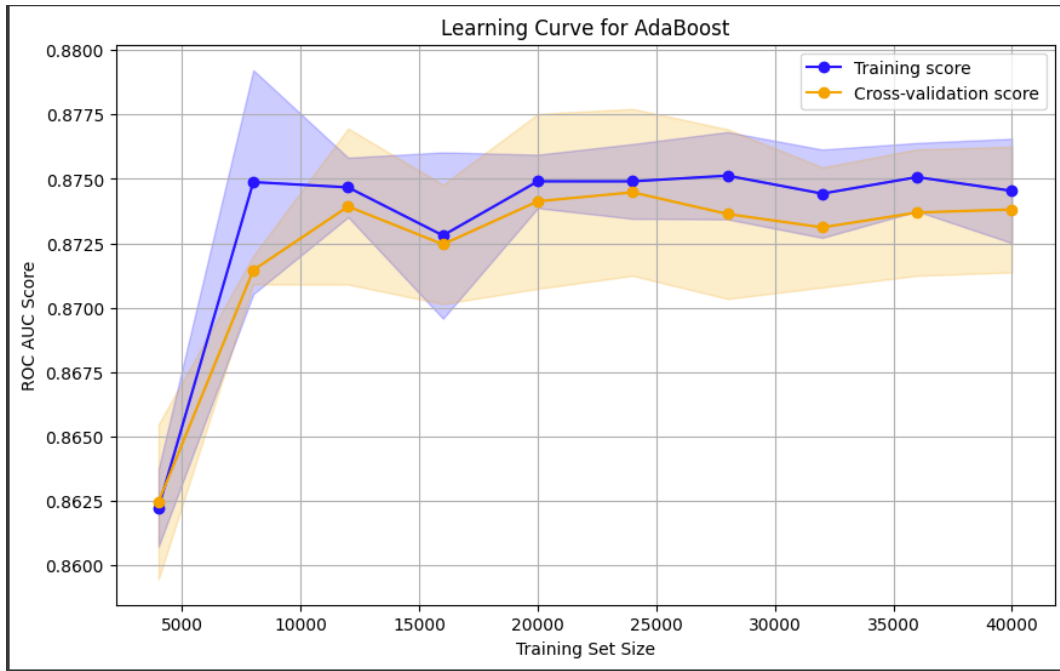


Tableau récapitulatif des résultats

Model	Seuil	Sensitivity	Specificity	F1 Score	AUC	Accuracy
XGBoost	0.39	0.8481	0.7798	0.6704	0.8585	0.7965
Gradient Boost	0.32	0.8462	0.7748	0.6653	0.8554	0.7922
AdaBoost	0.49	0.8461	0.7760	0.6662	0.8538	0.7931
Logistic Regression	0.365	0.8851	0.7621	0.6751	0.8506	0.7921

Table 5: Résultats des modèles avec leurs seuils et métriques

En terme de performance (**AUC** et **F1 score**) : Gradient boost < AdaBoost < XGBoost < Logistic regression

3.2.5 Choix du modèle final 2ème méthode

Le modèle qu'on retiendra au final est : **logistic regression de la deuxième méthode** (cf. paragraphe précédent). Mais nous aurions pu retenir le XGBoost. En effet, le XGBoost donne aussi de bonnes performances. Nous avons d'ailleurs fait les prédictions avec les deux modèles. Et sur les 30000 observations de la base de **test**, nous n'obtenons que 1344 prédictions différentes, soit 4,5%, ce qui vient confirmer notre idée.

3.2.6 Commentaires sur la 2ème méthode

Cette méthode nous a permis de comprendre l'importance du seuil de la matrice de confusion.

4 Conclusion

4.1 Modèle retenu

Le modèle retenu pour cette étude est le **Gradient Boosting Classifier** de la première méthode. Il s'est avéré être le plus performant parmi les approches testées. Il combine à la fois la puissance des arbres de décision et la capacité d'optimisation via le boosting, offrant ainsi une modélisation robuste adaptée à nos données.

Après un processus rigoureux d'entraînement, d'optimisation des hyperparamètres et d'évaluation, ce modèle a démontré sa capacité à équilibrer efficacement les métriques de performance, notamment :

- Un **F1-Score** optimal de **0.68**, maximisé grâce à un ajustement du seuil de décision (**0.60**), garantissant un compromis judicieux entre précision et rappel.
- Une **AUC (Area Under the Curve)** de **0.83**, confirmant une bonne capacité de discrimination entre les classes.
- Une précision accrue pour la classe dominante tout en minimisant les erreurs sur la classe minoritaire.

L'ajustement des paramètres clés, tels que la profondeur maximale des arbres, le taux d'apprentissage, et le nombre d'itérations, a permis d'éviter le sur-apprentissage, tout en tirant parti des informations complexes contenues dans les données.

De plus, l'utilisation d'un pipeline intégré avec des techniques avancées comme le **PCA** (Analyse en Composantes Principales) pour la réduction de la dimensionnalité, et la normalisation des données a contribué à améliorer la stabilité et la performance globale du modèle.

Enfin, le modèle a été validé à l'aide d'une **courbe d'apprentissage**, illustrant la capacité de généralisation et la convergence vers des performances stables avec une augmentation des données d'entraînement.

Table 6: Résumé des paramètres du modèle, pipeline et rééchantillonnage

Composant	Description et Paramètres
Modèle	Gradient Boosting Classifier (<code>n_estimators=50</code> , <code>max_depth=5</code> , <code>learning_rate=0.2</code>)
Pipeline	• PCA (<code>n_components=0.975</code>) : réduction de dimensionnalité • StandardScaler : normalisation des variables • PolynomialFeatures (<code>degree=2</code>) : génération de nouvelles variables polynomiales • SelectKBest (<code>k=15</code>) : sélection des 15 meilleures variables explicatives
Rééchantillonnage	Random UnderSampling (<code>sampling_strategy=0.9</code>) : équilibrage des classes en sous-échantillonnant la classe majoritaire

4.2 Limites du modèle et optimisations possibles

Au cours de ce projet, nous avons testé plusieurs modèles de Machine Learning dans le but d'améliorer les performances de prédiction sur notre jeu de données. Parmi ces modèles, nous nous sommes particulièrement attardés sur **GradientBoosting** couplé avec la technique de rééchantillonnage **RandomUnderSampling**.

L'objectif principal était d'optimiser ce modèle en utilisant des techniques comme **RandomizedSearchCV**, qui consiste à explorer un espace d'hyperparamètres plus large et de manière plus aléatoire que **GridSearchCV**. Cela permet de trouver des combinaisons d'hyperparamètres potentiellement meilleures sans être limité par une grille fixe. Nous avons effectué plusieurs recherches sur un large éventail de paramètres, incluant le **learning rate**, le **n_estimators**, ainsi que le **max_depth** de l'arbre de décision.

Cependant, malgré **de longues recherches**, les résultats obtenus n'ont pas été concluants. Les modèles optimisés via **RandomizedSearchCV** n'ont pas permis d'améliorer significativement les performances par rapport aux configurations de base. En effet, les résultats obtenus ont montré des gains marginaux en termes de **F1-score** et d'**AUC**, mais ces gains ne justifiaient pas le temps et les ressources investis dans ces recherches.

Cela souligne que, bien que **GradientBoosting** avec **RandomUnderSampling** semble prometteur sur le papier, l'optimisation des hyperparamètres dans ce contexte particulier n'a pas permis de réaliser une amélioration substantielle de la performance du modèle. Cela pourrait être dû à la nature du déséquilibre des classes dans notre jeu de données, ou à d'autres facteurs sous-jacents liés à la complexité du modèle.

Nous regrettons de ne pas avoir pu exploiter pleinement des méthodes plus avancées d'optimisation des hyperparamètres, comme la **Random Search**. Les contraintes de temps et de calcul ont limité la possibilité de tester un plus large éventail de configurations, ce qui aurait potentiellement permis d'atteindre des performances encore meilleures.

Enfin, le modèle a été validé à l'aide d'une **courbe d'apprentissage**, illustrant la capacité de généralisation et la convergence vers des performances stables avec une augmentation des données d'entraînement.

Nous avons vu, notamment avec nos travaux pour la seconde méthode, que la détermination du seuil de décision pour la matrice de confusion influe de manière importante sur la valeur des métriques. Il est par défaut paramétré à 0.5 mais il peut être intéressant de le modifier. Dans notre cas, la baisser, signifie qu'on souhaite une précision accrue sur les individus qui vont signer ce contrat. C'est un parti pris.

Enfin, nous avons travaillé sur Google collab et nous rendons un fichier code .ipynb. Cependant, il ne contient qu'une seule cellule, en **python**. Nous avons mis 3 commentaires "BALISE xx" aux endroits pour mettre l'emplacement des fichiers.

5 Annexe

Méthode 1: Code pour le modèle final

```
1  # -----
2  # 1. Importation des bibliothèques et des données
3  # -----
4  import pandas as pd
5  import numpy as np
6  from sklearn.cluster import KMeans
7  from sklearn.preprocessing import StandardScaler
8  from sklearn.model_selection import train_test_split
9  from sklearn.pipeline import Pipeline
10 from sklearn.ensemble import GradientBoostingClassifier
11 from sklearn.metrics import classification_report, precision_recall_curve, f1_score
12     , roc_auc_score, confusion_matrix
13 import matplotlib.pyplot as plt
14 from sklearn.decomposition import PCA
15 from sklearn.preprocessing import PolynomialFeatures
16 from sklearn.feature_selection import SelectKBest, f_classif
17 from sklearn.pipeline import make_pipeline
18 from sklearn.metrics import accuracy_score, auc, roc_curve, precision_score,
19     recall_score
20 from sklearn.model_selection import GridSearchCV
21 from imblearn.over_sampling import RandomOverSampler
22 from imblearn.pipeline import Pipeline as ImbPipeline
23 import warnings
24
25 warnings.filterwarnings("ignore")
26
27 # Monture de Google Drive
28 from google.colab import drive
29 drive.mount('/content/drive')
30
31 # Chemin du fichier
32 file_path = '/content/drive/MyDrive/PROJET MACHINE LEARNING/train_2.csv'
33
34 df = pd.read_csv(file_path, sep=';')
35
36 # -----
37 # 2. Définition des fonctions pour le prétraitement
38 # -----
39
40 def categorize_vintage(df):
41     if 'Vintage' not in df.columns:
42         return df
43     df['Vintage'] = df['Vintage'].apply(lambda x: max(x, 0) if pd.notnull(x) else
44         0)
45     df['Vintage_Years'] = df['Vintage'] / 365
46     max_vintage = df['Vintage_Years'].max()
47     bins = [0, 0.87, max(0.87 + 1e-6, max_vintage + 1e-6)]
48     labels = ['< 0.87 years', '> 0.87 years']
49     df['Vintage_Cat'] = pd.cut(df['Vintage_Years'], bins=bins, labels=labels, right
50         =True, include_lowest=True)
51     return df
```

```
48 def categorize_annual_premium(df):
49     max_premium = df['Annual_Premium'].max()
50     bins = [0, 10000, 55000, 60000, max_premium]
51     bins = sorted(list(set(bins)))
52     labels = ['Low', 'Medium', 'High', 'Very High']
53     if len(bins) < 5:
54         labels = labels[:len(bins) - 1]
55     df['Annual_Premium_Category'] = pd.cut(df['Annual_Premium'], bins=bins, labels=
        labels, right=False, include_lowest=True, duplicates='drop')
56     return df
57
58 def transform_log_annual_premium(df):
59     df['Annual_Premium_Log'] = np.log1p(df['Annual_Premium'])
60     return df
61
62 def categorize_age(df):
63     bins = [18, 27, 37, 50, 62, 72, 83, 110]
64     labels = ['18-27', '28-37', '38-50', '51-62', '63-72', '73-83', '84-110']
65     df['Age_Cat'] = pd.cut(df['Age'], bins=bins, labels=labels, right=True,
        include_lowest=True)
66     return df
67
68 def categorize_int_policy_sales_channel(df, threshold=50):
69     channel_counts = df['Policy_Sales_Channel'].value_counts()
70     df['Policy_Sales_Channel'] = df['Policy_Sales_Channel'].apply(lambda x: x if
        channel_counts[x] >= threshold else -1)
71     return df
72
73 def categorize_int_region_code(df, threshold=250):
74     region_counts = df['Region_Code'].value_counts()
75     df['Region_Code'] = df['Region_Code'].apply(lambda x: x if region_counts[x] >=
        threshold else -1)
76     return df
77
78 def kmeans_region_policy_premium(df, n_clusters=10):
79     columns = ['Region_Code', 'Policy_Sales_Channel', 'Annual_Premium']
80     available_columns = [col for col in columns if col in df.columns]
81     clustering_data = df[available_columns].fillna(0)
82     kmeans = KMeans(n_clusters=n_clusters, random_state=42)
83     df['Cluster'] = kmeans.fit_predict(clustering_data)
84     return df
85
86 def encode_columns(df):
87     code_vehicle_damage = {'Yes': 1, 'No': 0}
88     code_gender = {'Male': 1, 'Female': 0}
89     code_vehicle_age = {'< 1 Year': 0, '1-2 Year': 1, '> 2 Years': 2}
90     df['Vehicle_Damage'] = df['Vehicle_Damage'].map(code_vehicle_damage)
91     df['Gender'] = df['Gender'].map(code_gender)
92     df['Vehicle_Age'] = df['Vehicle_Age'].map(code_vehicle_age)
93     df = pd.get_dummies(df, columns=['Age_Cat', 'Annual_Premium_Category', 'Cluster
        ', 'Vintage_Cat'], drop_first=True)
94     return df
95
96 def preprocessing(df):
97     df = categorize_vintage(df)
```



```
98     df = categorize_age(df)
99     df = categorize_annual_premium(df)
100    df = transform_log_annual_premium(df)
101    df = categorize_int_policy_sales_channel(df, threshold=25)
102    df = categorize_int_region_code(df, threshold=250)
103    df = kmeans_region_policy_premium(df, n_clusters=10)
104    df = encode_columns(df)
105    df.drop(['id', 'Age', 'Vintage', 'Annual_Premium', 'Region_Code', '
106            Policy_Sales_Channel'], axis=1, inplace=True, errors='ignore')
107    return df
108
109    # -----
110    # 3. Pr eparation des donn es d'entra nement
111    # -----
112    train, test = train_test_split(df, test_size=0.2, random_state=42)
113
114    X_train = preprocessing(train)
115    X_test = preprocessing(test)
116
117    y_train = X_train['Response']
118    X_train = X_train.drop('Response', axis=1)
119
120    y_test = X_test['Response']
121    X_test = X_test.drop('Response', axis=1)
122
123    X_train, X_test = X_train.align(X_test, join='left', axis=1)
124
125    model = ImbPipeline([
126        ('pca', PCA(n_components=0.975)),
127        ('scaler', StandardScaler()),
128        ('poly', PolynomialFeatures(degree=2, include_bias=False)),
129        ('selector', SelectKBest(f_classif, k=15)),
130        ('sampler', RandomOverSampler(random_state=42, sampling_strategy=0.9)),
131        ('classifier', GradientBoostingClassifier(random_state=42, n_estimators=50))
132    ])
133
134    def evaluation_model_with_pr_curve_and_threshold(model):
135        # Entra nement du mod le
136        model.fit(X_train, y_train)
137
138        # Pr dictions et scores
139        y_pred = model.predict(X_test)
140        y_scores = model.predict_proba(X_test)[:, 1]
141
142        # Affichage des m triques classiques
143        print("F1 score : ", f1_score(y_test, y_pred))
144        print('AUC score : ', roc_auc_score(y_test, y_pred))
145        print(confusion_matrix(y_test, y_pred))
146        print(classification_report(y_test, y_pred))
147
148        # Calcul de la Precision-Recall Curve
149        precision, recall, thresholds = precision_recall_curve(y_test, y_scores)
150
151        # Calcul du F1-score pour chaque seuil
152        f1_scores = 2 * (precision * recall) / (precision + recall)
```

```
152     best_index = np.argmax(f1_scores)
153     best_threshold = thresholds[best_index]
154
155     print(f"Meilleur seuil bas sur le F1-score : {best_threshold:.2f}")
156     print(f"F1-score associ : {f1_scores[best_index]:.2f}")
157
158     # Visualisation de la Precision-Recall Curve
159     plt.figure(figsize=(10, 6))
160     plt.plot(recall, precision, label='Precision-Recall Curve')
161     plt.scatter(recall[best_index], precision[best_index], color='red', label=f"
162                 Meilleur seuil (F1 = {f1_scores[best_index]:.2f})")
163     plt.xlabel('Recall')
164     plt.ylabel('Precision')
165     plt.title('Precision-Recall Curve avec meilleur seuil')
166     plt.legend()
167     plt.grid()
168     plt.show()
169
170 evaluation_model_with_pr_curve_and_threshold(model)
171
172
173 # -----
174 # 4. Prediction
175 # -----
176
177 # Charger les donn es pr dire
178 pred_file = '/content/drive/MyDrive/PROJET MACHINE LEARNING/test.csv'
179 X_pred = pd.read_csv(pred_file, sep=',')
180
181 # Stocker l'identifiant 'id' avant le pr traitement
182 id_column = X_pred['id']
183
184 # Pr traitement des donn es pr dire
185 X_pred = preprocessing(X_pred)
186 X_pred = X_pred.align(X_train, join='left', axis=1)[0] # Aligement avec X_train
187 X_pred.fillna(0, inplace=True) # Remplir les valeurs manquantes
188
189 # G n rer les pr dictions pour les nouvelles donn es
190 y_prob_pred = model.predict_proba(X_pred)[: , 1] # Probabilit s pour la classe
191           positive
192 best_threshold = 0.51 # Seuil pour la classification, A regarder
193 X_pred['Probability'] = y_prob_pred
194 X_pred['Prediction'] = (y_prob_pred >= best_threshold).astype(int) # Ajouter les
195                       pr dictions binaires
196
197 # R int grer la colonne 'id'
198 X_pred['id'] = id_column
199
200 # Garder uniquement les colonnes id, Probability et Prediction
201 output_df = X_pred[['id', 'Probability', 'Prediction']]
202
203 # Sauvegarder les pr dictions
204 output_file = '/content/drive/MyDrive/fichier_avec_predictions.csv'
205 output_df.to_csv(output_file, index=False, sep=';')
```

```
204 print(f"Fichier avec pr ddictions enregistr      : {output_file}")
205
206 # -----
207 # 5. Fichier CSV
208 # -----
209
210 file_predictions = '/content/drive/MyDrive/fichier_avec_predictions.csv'
211
212 df = pd.read_csv(file_predictions, sep=';')
213
214 # On ne garde que les variable id, probability et prediction
215 df = df[['id', 'Probability', 'Prediction']]
216
217 # Combien de pr ddictions positives et n gatives
218 value_counts = df['Prediction'].value_counts(normalize=True)
219 print(value_counts)
```

Méthode 2 (non retenue): Code

```
1
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 %matplotlib inline
6 import seaborn as sns
7 import warnings
8 warnings.filterwarnings('ignore')
9 from scipy.stats import chi2_contingency
10 import itertools
11
12 from google.colab import drive
13 drive.mount('/content/drive')
14
15 file_path = '/content/drive/My Drive/PROJET MACHINE LEARNING/test.csv'
16 df1 = pd.read_csv(file_path, sep=',')
17
18 file_path_2 = '/content/drive/My Drive/PROJET MACHINE LEARNING/train_2.csv'
19 df2 = pd.read_csv(file_path_2, sep=';')
20
21 # Compter les réponses positives par région dans df2
22 region_response_count = df2[df2['Response'] == 1].groupby('Region_Code').size()
23
24 # Trier les régions en fonction du nombre de réponses positives en ordre
   d croissant
25 sorted_regions = region_response_count.sort_values(ascending=False)
26
27 # Déterminer les catégories de regroupement pour les régions
28 top_5_regions = sorted_regions.index[:5] # Les 5 régions avec le plus de
   réponses positives
29 next_5_regions = sorted_regions.index[5:10] # Les 5 suivantes
30 remaining_regions = sorted_regions.index[10:] # Le reste des régions
31
32 # Fonction d'assignation de groupe de région
33 def assign_region_group(region):
34     if region in top_5_regions:
35         return 1 # Groupe 1 : Les 5 premières régions
36     elif region in next_5_regions:
37         return 2 # Groupe 2 : Les 5 suivantes
38     else:
39         return 3 # Groupe 3 : Toutes les autres régions
40
41 # Appliquer la fonction pour créer la colonne de regroupement des régions dans
   df2
42 df2['Region_Group'] = df2['Region_Code'].apply(assign_region_group)
43
44 # Afficher le DataFrame avec la nouvelle colonne de regroupement des régions pour
   vérification
45 #print(df2[['Region_Code', 'Region_Group']].head(15))
46
47 import pandas as pd
48 import numpy as np
49
```

```
50 # 1. Transformation de la colonne 'Vintage' en ann es et cat gorisation par
    tranches
51 df2['Vintage'] = df2['Vintage'] / 365.25 # Conversion en ann es
52
53 def categorize_age(age):
54     if 18 <= age < 30: #La jeunesse
55         return 1
56     elif 30 <= age < 40: #On devient p re de familles
57         return 2
58     elif 40 <= age < 50: #On devient responsables, cadre etc.
59         return 3
60     elif 50 <= age < 60: #S nior
61         return 4
62     else:
63         return 5
64
65 df2['Age'] = df2['Age'].apply(categorize_age)
66
67 def categorize_vintage(vintage):
68     if vintage < 0.25: # Moins d'un trimestre (3 mois)
69         return 1
70     elif 0.25 <= vintage < 0.5: # Entre 3 mois et 6 mois
71         return 2
72     elif 0.5 <= vintage < 0.75: # Entre 6 mois et 9 mois
73         return 3
74     elif 0.75 <= vintage < 1: # Entre 9 mois et 1 an
75         return 4
76     else: # Plus d'un an
77         return 5
78
79 policy_response_count = df2[df2['Response'] == 1].groupby('Policy_Sales_Channel').
    size()
80
81 # tape 2 : Trier les canaux par le nombre de r ponses positives
82 sorted_channels = policy_response_count.sort_values(ascending=False)
83
84 # Cat goriser les canaux en trois groupes
85 top_2_channels = sorted_channels.index[:2]
86 next_6_channels = sorted_channels.index[2:8]
87
88 #Cat goriser annual premium
89 import pandas as pd
90 import numpy as np
91
92 # Fonction pour cat goriser Annual_Premium
93 def categorize_annual_premium(premium):
94     max_premium = df2['Annual_Premium'].max() # Utilisation de df2
95     bins = [0, 10000, 55000, 60000, max_premium]
96     bins = sorted(list(set(bins)))
97     labels = ['Low', 'Medium', 'High', 'Very High']
98     if len(bins) < 5:
99         labels = labels[:len(bins) - 1]
100     # Cat gorisation de la prime
101     return pd.cut([premium], bins=bins, labels=labels, right=False, include_lowest=
        True, duplicates='drop')[0]
```

```
102
103 # Fonction pour appliquer la transformation logarithmique
104 def transform_log_annual_premium(premium):
105     return np.log1p(premium)
106
107 # Appliquer les transformations
108 df2['Annual_Premium_Category'] = df2['Annual_Premium'].apply(
109     categorize_annual_premium)
110 df2['Annual_Premium_Log'] = df2['Annual_Premium'].apply(
111     transform_log_annual_premium)
112
113 # Fonction de regroupement des canaux
114 def assign_channel_group(channel):
115     if channel in top_2_channels:
116         return 1 # Groupe 1 : les deux canaux avec le plus de r ponses positives
117     elif channel in next_6_channels:
118         return 2 # Groupe 2 : les six suivants
119     else:
120         return 3 # Groupe 3 : le reste
121
122 # Appliquer la transformation
123 df2['Channel_Group'] = df2['Policy_Sales_Channel'].apply(assign_channel_group)
124
125 df2['Vintage'] = df2['Vintage'].apply(categorize_vintage)
126
127 # 2. Transformation de 'Gender' en valeurs binaires : 0 pour 'Female', 1 pour 'Male'
128 df2['Gender'] = df2['Gender'].map({'Female': 0, 'Male': 1}).astype('category')
129
130 # 3. Transformation de 'Driving_License' et 'Previously_Insured' en cat gories
131 df2['Driving_License'] = df2['Driving_License'].astype('category')
132 df2['Previously_Insured'] = df2['Previously_Insured'].astype('category')
133
134 # 4. Transformation de 'Vehicle_Age' en valeurs num riques : 0 pour '< 1 Year', 1
135 # pour '1-2 Year', 2 pour '> 2 Years'
136 df2['Vehicle_Age'] = df2['Vehicle_Age'].map({'< 1 Year': 0, '1-2 Year': 1, '> 2
137     Years': 2}).astype('category')
138
139 # 5. Transformation de 'Vehicle_Damage' en valeurs binaires : 1 pour 'Yes', 0 pour
140 # 'No'
141 df2['Vehicle_Damage'] = df2['Vehicle_Damage'].map({'Yes': 1, 'No': 0}).astype('
142     category')
143
144 # Installer la biblioth que prince si elle n'est pas install e
145 try:
146     import prince
147 except ImportError:
148     !pip install prince
149     import prince
150
151 # Importer les autres biblioth ques n cessaires
152 import pandas as pd
153 import matplotlib.pyplot as plt
```

```
150 # Sélectionner les colonnes pour l'AFCM
151 columns_for_afcm = [
152     'Region_Group', 'Vintage', 'Gender', 'Driving_License',
153     'Previously_Insured', 'Vehicle_Age', 'Vehicle_Damage', 'Age', 'Channel_Group',
154     'Annual_Premium_Category'
155 ]
156 # Filtrer le DataFrame pour ne garder que les colonnes pertinentes pour l'AFCM
157 # Remplacez 'df2' par le nom actuel de votre DataFrame si différent
158 df_afcm = df2[columns_for_afcm].copy()
159
160 # S'assurer que toutes les colonnes sont bien au format 'category' si ce n'est pas
161 # le cas
162 for col in columns_for_afcm:
163     df_afcm[col] = df_afcm[col].astype('category')
164
165 # Initialiser et ajuster le modèle AFCM
166 afcm = prince.MCA(n_components=2, n_iter=3, check_input=True, copy=True)
167 afcm = afcm.fit(df_afcm)
168
169 # Extraire les coordonnées des colonnes pour le graphique
170 column_coordinates = afcm.column_coordinates(df_afcm)
171
172 # Créer le graphique
173 plt.figure(figsize=(10, 8))
174 plt.scatter(column_coordinates[0], column_coordinates[1], color='blue', s=100)
175
176 # Ajouter les étiquettes des variables pour chaque point
177 for i, variable in enumerate(column_coordinates.index):
178     plt.annotate(variable, (column_coordinates[0][i], column_coordinates[1][i]),
179                  fontsize=12)
180
181 # Supposons que df2 est votre DataFrame contenant les colonnes à encoder
182
183 # Encoder Driving_License
184 df2['Driving_License_0'] = (df2['Driving_License'] == 0).astype(int)
185 df2['Driving_License_1'] = (df2['Driving_License'] == 1).astype(int)
186
187 # Encoder Age
188 df2['Age_1'] = (df2['Age'] == 1).astype(int)
189 df2['Age_2'] = (df2['Age'] == 2).astype(int)
190 df2['Age_3'] = (df2['Age'] == 3).astype(int)
191 df2['Age_4'] = (df2['Age'] == 4).astype(int)
192 df2['Age_5'] = (df2['Age'] == 5).astype(int)
193
194 # Encoder Region_Group
195 df2['Region_Group_1'] = (df2['Region_Group'] == 1).astype(int)
196 df2['Region_Group_2'] = (df2['Region_Group'] == 2).astype(int)
197 df2['Region_Group_3'] = (df2['Region_Group'] == 3).astype(int)
198
199 # Encoder Channel_Group
200 df2['Channel_Group_1'] = (df2['Channel_Group'] == 1).astype(int)
201 df2['Channel_Group_2'] = (df2['Channel_Group'] == 2).astype(int)
202 df2['Channel_Group_3'] = (df2['Channel_Group'] == 3).astype(int)
```

```
202
203 # Encoder Vehicle_Damage
204 df2['Vehicle_Damage_0'] = (df2['Vehicle_Damage'] == 0).astype(int)
205 df2['Vehicle_Damage_1'] = (df2['Vehicle_Damage'] == 1).astype(int)
206
207 # Encoder Vehicle_Age
208 df2['Vehicle_Age_0'] = (df2['Vehicle_Age'] == 0).astype(int)
209 df2['Vehicle_Age_1'] = (df2['Vehicle_Age'] == 1).astype(int)
210 df2['Vehicle_Age_2'] = (df2['Vehicle_Age'] == 2).astype(int)
211
212 # Encoder Previously_Insured
213 df2['Previously_Insured_0'] = (df2['Previously_Insured'] == 0).astype(int)
214 df2['Previously_Insured_1'] = (df2['Previously_Insured'] == 1).astype(int)
215
216 # Encoder Vintage
217 df2['Vintage_1'] = (df2['Vintage'] == 1).astype(int)
218 df2['Vintage_2'] = (df2['Vintage'] == 2).astype(int)
219 df2['Vintage_3'] = (df2['Vintage'] == 3).astype(int)
220 df2['Vintage_4'] = (df2['Vintage'] == 4).astype(int)
221
222
223 df2['Annual_Premium_Low']=(df2['Annual_Premium']=='Low')
224 df2['Annual_Premium_very_high']=(df2['Annual_Premium']=='Very High')
225 # Suppression des colonnes originales si n cessaire
226 df2 = df2.drop(columns=['Driving_License', 'Age', 'Region_Group', 'Channel_Group',
227                        'Vehicle_Damage', 'Vehicle_Age', 'Previously_Insured', '
228                        Vintage'], errors='ignore')
229
230
231
232
233
234 df2['Gender_0'] = (df2['Gender'] == 0).astype(int) # 1 pour Female
235 df2['Gender_1'] = (df2['Gender'] == 1).astype(int) # 1 pour Male
236
237
238 import pandas as pd
239 import numpy as np
240 from sklearn.linear_model import LogisticRegression
241 from sklearn.model_selection import StratifiedKFold
242 from sklearn.metrics import roc_auc_score, f1_score, confusion_matrix
243
244 # S lection des variables ind pendantes et de la variable cible
245 X = df2[[ # Assurez-vous que ces colonnes existent dans df2
246          'Driving_License_0', 'Age_1', 'Age_2', 'Age_3', 'Age_4',
247          'Vehicle_Damage_1', 'Channel_Group_2', 'Previously_Insured_0',
248          'Region_Group_2', 'Region_Group_1', 'Gender_0', 'Vintage_1', 'Vintage_2',
249          'Vintage_3', 'Vehicle_Age_2', 'Channel_Group_3', 'Age_2', 'Age_3', 'Age_4', '
250          Annual_Premium_very_high', 'Annual_Premium_Low'
251 ]]
252 y = df2['Response']
253
254 # Initialiser la validation crois e
255 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```



```
255
256 sensitivity_list = []
257 specificity_list = []
258 f1_score_list = []
259 auc_list = []
260 accuracy_list = [] # Liste pour stocker les valeurs de l'accuracy
261
262 # Validation croise e
263 for train_index, test_index in skf.split(X, y):
264     X_train, X_test = X.iloc[train_index], X.iloc[test_index]
265     y_train, y_test = y.iloc[train_index], y.iloc[test_index]
266
267     # Cr er et ajuster le mod le
268     model = LogisticRegression(solver='lbfgs') # Utilisation d'un solveur
           compatible avec des petits chantillons
269     model.fit(X_train, y_train)
270
271     # Pr dictions
272     y_probs = model.predict_proba(X_test)[: , 1] # Probabilit s pour la classe 1
273
274     y_pred = (y_probs >= 0.5).astype(int) # Faire bouger les seuils pour prdre
           celui qui donne le meilleur F1 score
275
276     # valuation des performances
277     tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
278
279     sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0
280     specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
281     f1 = f1_score(y_test, y_pred)
282     auc = roc_auc_score(y_test, y_probs)
283
284     # Calcul de l'accuracy
285     accuracy = (tp + tn) / (tp + tn + fp + fn)
286
287     # Ajouter les r sultats des m triques dans leurs listes respectives
288     sensitivity_list.append(sensitivity)
289     specificity_list.append(specificity)
290     f1_score_list.append(f1)
291     auc_list.append(auc)
292     accuracy_list.append(accuracy) # Ajouter l'accuracy
293
294 # Moyennes des performances
295 average_sensitivity = np.mean(sensitivity_list)
296 average_specificity = np.mean(specificity_list)
297 average_f1_score = np.mean(f1_score_list)
298 average_auc = np.mean(auc_list)
299 average_accuracy = np.mean(accuracy_list) # Moyenne de l'accuracy
300
301 # Affichage des r sultats
302 #print(f"Sensitivity: {average_sensitivity:.4f}")
303 #print(f"Specificity: {average_specificity:.4f}")
304 #print(f"F1 Score: {average_f1_score:.4f}")
305 #print(f"AUC: {average_auc:.4f}")
306 #print(f"Accuracy: {average_accuracy:.4f}") # Afficher l'accuracy
307
```

```
308
309
310
311
312 import pandas as pd
313 import numpy as np
314
315
316 # Appliquer la fonction pour cr er la colonne de regroupement des r gions dans
    df1
317 df1['Region_Group'] = df1['Region_Code'].apply(assign_region_group)
318
319 # 1. Transformation de la colonne 'Vintage' en ann es et cat gorisation par
    tranches
320 df1['Vintage'] = df1['Vintage'] / 365.25 # Conversion en ann es
321
322 df1['Age'] = df1['Age'].apply(categorize_age)
323
324 # Cat goriser les canaux en trois groupes
325 top_2_channels = sorted_channels.index[:2]
326 next_6_channels = sorted_channels.index[2:8]
327
328 # Fonction pour cat goriser Annual_Premium
329 def categorize_annual_premium(premium):
330     max_premium = df1['Annual_Premium'].max() # Utilisation de df1
331     bins = [0, 10000, 55000, 60000, max_premium]
332     bins = sorted(list(set(bins)))
333     labels = ['Low', 'Medium', 'High', 'Very High']
334     if len(bins) < 5:
335         labels = labels[:len(bins) - 1]
336     # Cat gorisation de la prime
337     return pd.cut([premium], bins=bins, labels=labels, right=False, include_lowest=
        True, duplicates='drop')[0]
338
339 # Appliquer les transformations
340 df1['Annual_Premium_Category'] = df1['Annual_Premium'].apply(
    categorize_annual_premium)
341 df1['Annual_Premium_Log'] = df1['Annual_Premium'].apply(
    transform_log_annual_premium)
342
343 # Afficher les colonnes pour v rification
344
345 # Fonction de regroupement des canaux
346 def assign_channel_group(channel):
347     if channel in top_2_channels:
348         return 1 # Groupe 1 : les deux canaux avec le plus de r ponses positives
349     elif channel in next_6_channels:
350         return 2 # Groupe 2 : les six suivants
351     else:
352         return 3 # Groupe 3 : le reste
353
354 # Appliquer la transformation
355 df1['Channel_Group'] = df1['Policy_Sales_Channel'].apply(assign_channel_group)
356
357 df1['Vintage'] = df1['Vintage'].apply(categorize_vintage)
```

```
358
359 # 2. Transformation de 'Gender' en valeurs binaires : 0 pour 'Female', 1 pour 'Male'
360 df1['Gender'] = df1['Gender'].map({'Female': 0, 'Male': 1}).astype('category')
361
362 # 3. Transformation de 'Driving_License' et 'Previously_Insured' en categories
363 df1['Driving_License'] = df1['Driving_License'].astype('category')
364 df1['Previously_Insured'] = df1['Previously_Insured'].astype('category')
365
366 # 4. Transformation de 'Vehicle_Age' en valeurs numériques : 0 pour '< 1 Year', 1
    pour '1-2 Year', 2 pour '> 2 Years'
367 df1['Vehicle_Age'] = df1['Vehicle_Age'].map({'< 1 Year': 0, '1-2 Year': 1, '> 2
    Years': 2}).astype('category')
368
369 # 5. Transformation de 'Vehicle_Damage' en valeurs binaires : 1 pour 'Yes', 0 pour
    'No'
370 df1['Vehicle_Damage'] = df1['Vehicle_Damage'].map({'Yes': 1, 'No': 0}).astype('
    category')
371
372 # Supposons que df1 est votre DataFrame contenant les colonnes encoder
373
374 # Encoder Driving_License
375 df1['Driving_License_0'] = (df1['Driving_License'] == 0).astype(int)
376 df1['Driving_License_1'] = (df1['Driving_License'] == 1).astype(int)
377
378 # Encoder Age
379 df1['Age_1'] = (df1['Age'] == 1).astype(int)
380 df1['Age_2'] = (df1['Age'] == 2).astype(int)
381 df1['Age_3'] = (df1['Age'] == 3).astype(int)
382 df1['Age_4'] = (df1['Age'] == 4).astype(int)
383 df1['Age_5'] = (df1['Age'] == 5).astype(int)
384
385 # Encoder Region_Group
386 df1['Region_Group_1'] = (df1['Region_Group'] == 1).astype(int)
387 df1['Region_Group_2'] = (df1['Region_Group'] == 2).astype(int)
388 df1['Region_Group_3'] = (df1['Region_Group'] == 3).astype(int)
389
390 # Encoder Channel_Group
391 df1['Channel_Group_1'] = (df1['Channel_Group'] == 1).astype(int)
392 df1['Channel_Group_2'] = (df1['Channel_Group'] == 2).astype(int)
393 df1['Channel_Group_3'] = (df1['Channel_Group'] == 3).astype(int)
394
395 # Encoder Vehicle_Damage
396 df1['Vehicle_Damage_0'] = (df1['Vehicle_Damage'] == 0).astype(int)
397 df1['Vehicle_Damage_1'] = (df1['Vehicle_Damage'] == 1).astype(int)
398
399 # Encoder Vehicle_Age
400 df1['Vehicle_Age_0'] = (df1['Vehicle_Age'] == 0).astype(int)
401 df1['Vehicle_Age_1'] = (df1['Vehicle_Age'] == 1).astype(int)
402 df1['Vehicle_Age_2'] = (df1['Vehicle_Age'] == 2).astype(int)
403
404 # Encoder Previously_Insured
405 df1['Previously_Insured_0'] = (df1['Previously_Insured'] == 0).astype(int)
406 df1['Previously_Insured_1'] = (df1['Previously_Insured'] == 1).astype(int)
407
```

```
408 # Encoder Vintage
409 df1['Vintage_1'] = (df1['Vintage'] == 1).astype(int)
410 df1['Vintage_2'] = (df1['Vintage'] == 2).astype(int)
411 df1['Vintage_3'] = (df1['Vintage'] == 3).astype(int)
412 df1['Vintage_4'] = (df1['Vintage'] == 4).astype(int)
413
414 df1['Annual_Premium_Low']=(df1['Annual_Premium']=='Low')
415 df1['Annual_Premium_very_high']=(df1['Annual_Premium']=='Very High')
416
417 df1['Gender_0'] = (df1['Gender'] == 0).astype(int) # 1 pour Female
418 df1['Gender_1'] = (df1['Gender'] == 1).astype(int) # 1 pour Male
419
420
421 # Assurez-vous que df1 contient toutes les colonnes nécessaires
422 X_new = df1[[ # Les mêmes colonnes que celles utilisées pour entraîner le
    mod le
423     'Driving_License_0', 'Age_1', 'Age_2', 'Age_3', 'Age_4',
424     'Vehicle_Damage_1', 'Channel_Group_2', 'Previously_Insured_0',
425     'Region_Group_2', 'Region_Group_1', 'Gender_0', 'Vintage_1', 'Vintage_2',
426     'Vintage_3', 'Vehicle_Age_2', 'Channel_Group_3', 'Age_2', 'Age_3', 'Age_4', '
        Annual_Premium_very_high', 'Annual_Premium_Low'
427 ]]
428
429 # Utiliser le modèle ajusté pour prédire les probabilités sur df1
430 y_probs_new = model.predict_proba(X_new)[: , 1] # Prédiction de probabilités
    pour la classe 1
431
432 # Si vous voulez convertir ces probabilités en prédictions binaires (1 ou 0) en
    utilisant un seuil :
433 threshold = 0.365 # Seuil utilisé précédemment
434 y_pred_new = (y_probs_new >= threshold).astype(int) # Conversion en 1 ou 0
435
436
437 # Générer le DataFrame avec les prédictions et les probabilités
438 prediction_df = pd.DataFrame({
439     'id': df1['id'], # Assurez-vous que la colonne 'id' existe dans df1
440     'probability': y_probs_new, # Probabilités prédites pour la classe 1
441     'response': y_pred_new # Prédiction binaire (0 ou 1) basée sur le seuil
442 })
443
444 # Sauvegarder ce DataFrame dans un fichier CSV
445 prediction_df.to_csv('prediction_1.csv', index=False) # Remplacez 1 par le numéro
    de votre groupe
446
447 from google.colab import drive
448 drive.mount('/content/drive')
449
450 # Générer et enregistrer le fichier CSV sur Google Drive
451 file_path = '/content/drive/MyDrive/PROJET MACHINE LEARNING/prediction_1.csv' #
    Modifiez le chemin selon votre besoin
452 prediction_df.to_csv(file_path, index=False)
453
454 print(f"Le fichier a été sauvegardé dans : {file_path}")
```